

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Sistem RAG pentru extragerea relațiilor între entități

Proiect de diplomă

prezentat ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Electronică și Telecomunicații*
programul de studii *Electronică Aplicată*

Conducător științific
Ș.l. Dr. Ing. Liviu-Daniel ȘTEFAN
Drd. Fiz. Alexandru IONIȚĂ

Absolvent
Andreas BAȘCHIR

Anul 2026

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Sistem RAG pentru extragerea relațiilor între entități*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității Naționale de Știință și Tehnologie POLITEHNICA București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul *Electronică și Telecomunicații*, programul de studii *Electronică Aplicată* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate.

Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Andreas BAȘCHIR

.....

Cuprins

Lista figurilor	7
Lista tabelelor	9
Lista acronimelor	11
Introducere	13
1. Studiu de literatură	15
1.1. Prezentarea domeniului lucrării de diplomă	15
1.1.1. Introducere	15
1.1.2. De la rețele neuronale la modele lingvistice mari	15
1.1.3. Problema: Cunoștințe statice care duc la halucinații	15
1.1.4. Soluția: Retrieval-Augmented Generation	16
1.1.5. RAG versus fine-tuning	17
1.1.6. Formularea problemei: Extragerea relațiilor între entități	17
1.1.7. Relevanță și aplicații	17
1.2. Taxonomie	18
1.2.1. Motivația definirii unei taxonomii	18
1.2.2. Criterii de clasificare	18
1.2.3. Categoriile principale	19
1.3. Arhitectura generală a unui sistem de tip RAG	20
1.3.1. Componentele sistemului	20
1.3.2. Fluxul de date	21
1.3.3. Exemplu de flux pentru extragerea relațiilor	22
1.3.4. Strategii de fragmentare	22
1.3.5. Modele de embedding și baze de date vectoriale	23
1.3.6. Construcția promptului și augmentarea contextului	23
1.3.7. Compromisuri în proiectarea unui sistem RAG	23
1.4. Metode și modele RAG	24
1.4.1. Naive RAG	24
1.4.2. Advanced RAG	24
1.4.3. Recuperarea densă: DPR și ColBERT	25
1.4.4. Hypothetical Document Embeddings (HyDE)	25
1.4.5. Reformularea și expansiunea interogărilor	25
1.4.6. REALM, Fusion-in-Decoder și RETRO	26
1.4.7. RAPTOR	26
1.4.8. Corrective RAG (CRAG)	26
1.4.9. Self-RAG	27
1.4.10. Adaptive-RAG și FLARE	27
1.4.11. GraphRAG și HippoRAG	27

1.4.12.	RAG pentru contexte lungi și aplicații specializate	28
1.4.13.	Sinteză comparativă	28
1.5.	Metrici de evaluare	28
1.5.1.	Metrici de recuperare	30
1.5.2.	Metrici de generare	30
1.5.3.	Metrici specifice sistemelor RAG	31
1.5.4.	Dificultăți în evaluarea extragerii relațiilor	31
1.6.	Baze de date	32
1.6.1.	Seturi de date pentru sisteme RAG	32
1.6.2.	Seturi de date pentru extragerea relațiilor	33
1.7.	Limitări și provocări	34
1.8.	Concluzii	34
2.	Rezultate experimentale	36
2.1.	Proiectarea sistemului	36
2.1.1.	Arhitectura generală	36
2.1.2.	Cerințe	36
2.1.3.	Tehnologii utilizate	37
2.1.4.	Configurația hardware	38
2.1.5.	Indexarea datelor	38
2.1.6.	Pipeline-ul Naive RAG	38
2.1.7.	Pipeline-ul Advanced RAG	39
2.1.8.	Interfața și fluxul utilizatorului	40
2.1.9.	Modulul de evaluare	41
2.1.10.	Procesul de dezvoltare	42
2.2.	Analiză cantitativă	43
2.2.1.	Configurarea experimentului	43
2.2.2.	Asigurarea unei comparații corecte	43
2.2.3.	Rezultate	44
2.2.4.	Semnificație statistică	44
2.2.5.	Stabilitatea rezultatelor pe seed-uri	45
2.3.	Analiză calitativă	46
2.3.1.	Compromisul dintre calitate și latență	46
2.3.2.	Unde ajută recuperarea hibridă	47
2.3.3.	Decalajul dintre recuperare și generare	48
2.3.4.	Constrângerea de grounding și refuzurile de a răspunde	48
2.3.5.	Observații calitative din interfața Streamlit	48
2.3.6.	Analiza erorilor	48
2.3.7.	Limitările demonstratorului	49
	Concluzii	50

Lista figurilor

1.1. Creșterea numărului de lucrări despre lingvistice mari (LLMs) publicate pe arXiv, 2017–2025.	16
1.2. Harta conceptuală a taxonomiei adoptate, structurată pe cele patru criterii relevante pentru extragerea relațiilor între entități [2, 6].	20
1.3. Arhitectura generală a unui sistem RAG: indexare offline (sus) și inferență online (jos) [2, 7].	22
2.1. Faza offline de indexare. Paragrafele unice din SQuAD sunt codificate o singură dată și depuse în indexul dens (ChromaDB) și în indexul rar (BM25), folosite în comun de ambele pipeline-uri.	37
2.2. Fluxul de procesare Naive RAG. Interogarea este codificată, cele mai apropiate cinci fragmente sunt căutate prin similaritate cosinus în ChromaDB și trimise direct modelului generativ, fără reformulare, recuperare hibridă sau reranking. . .	38
2.3. Fluxul de procesare Advanced RAG. Interogarea este reformulată, fiecare variantă este căutată dens și prin BM25, rezultatele sunt fuzionate prin Reciprocal Rank Fusion, re-scorate cu un cross-encoder, iar primele cinci fragmente sunt trimise modelului generativ.	39
2.4. Interfața comparativă a demonstratorului. Cele două coloane corespund pipeline-urilor Naive și Advanced, care răspund simultan la aceeași întrebare, cu timpii de generare afișați deasupra fiecărui răspuns. În acest exemplu, Naive RAG nu găsește răspunsul, în timp ce Advanced RAG recuperează fragmentul corect. . .	41
2.5. Exemplu de interogare cu termeni numerici. Naive RAG produce un răspuns numeric halucinat, deoarece similaritatea cosinus ratează valoarea exactă, în timp ce Advanced RAG găsește valoarea corectă prin recuperarea BM25.	42
2.6. Metrice de recuperare: Context Recall@5 și MRR pentru Naive RAG și Advanced RAG. Barele de eroare reprezintă deviația standard calculată pe cele 4 seed-uri.	44
2.7. Calitatea răspunsurilor: Answer F1 și Exact Match pentru Naive RAG și Advanced RAG.	45
2.8. Latența medie de procesare a interogării în milisecunde. Latența Advanced RAG include apelul LLM pentru expansiunea interogării, recuperarea hibridă paralelă și re-scorarea cu cross-encoder.	46
2.9. Vedere de ansamblu comparativă a metricilor de calitate (Context Recall@5, MRR, Answer F1, Exact Match). Latența nu este inclusă deoarece se află pe o scară diferită, în milisecunde față de intervalul de la 0 la 1.	47

Lista tabelelor

1.1. Sinteză comparativă a metodelor RAG prezentate și a relevanței lor pentru extragerea relațiilor între entități.	29
1.2. Comparatie a seturilor de date pentru evaluarea RAG și extragerea relațiilor [28–31, 33–38].	33
2.1. Tehnologiile utilizate în demonstrator	37
2.2. Parametrii pipeline-ului Advanced RAG	40
2.3. Rezultate comparative Naive RAG vs. Advanced RAG pe SQuAD ($N = 600$ per sistem, medie $\pm$ deviație standard pe 4 seed-uri)	44
2.4. Teste de semnificație statistică (Advanced RAG vs. Naive RAG, $N = 600$ perechi)	45
2.5. Context Recall@5 și MRR pe fiecare dintre cele 4 seed-uri (150 de întrebări per seed)	46

Lista acronimelor

API	Application Programming Interface
BM25	Best Matching 25 (funcție de scor pentru recuperare rară)
CRAG	Corrective Retrieval-Augmented Generation
DPR	Dense Passage Retrieval
EM	Exact Match
FiD	Fusion-in-Decoder
HNSW	Hierarchical Navigable Small World (graf de căutare aproximativă)
HyDE	Hypothetical Document Embeddings
IR	Information Retrieval (regăsirea informației)
LLM	Large Language Model (model lingvistic mare)
MRR	Mean Reciprocal Rank
NLP	Natural Language Processing (procesarea limbajului natural)
QA	Question Answering (răspuns la întrebări)
RAG	Retrieval-Augmented Generation (generare augmentată prin recuperare)
RAGAS	Retrieval-Augmented Generation Assessment
RDF	Resource Description Framework
RRF	Reciprocal Rank Fusion
SQuAD	Stanford Question Answering Dataset
VRAM	Video Random Access Memory (memorie video)

Introducere

Scopul lucrării de diplomă

Sistemele RAG (Retrieval-Augmented Generation) reprezintă o arhitectură de inteligență artificială care combină recuperarea informațiilor relevante din surse externe cu generarea de text de către modele de limbaj mari (LLMs - Large Language Models). În loc să se bazeze exclusiv pe cunoștințele încorporate în parametrii modelului la antrenare, un sistem RAG caută mai întâi documente relevante pentru întrebarea utilizatorului, apoi le folosește ca temei pentru răspuns.

RAG a fost propus ca soluție pentru natura statică a informațiilor reținute de modelele de limbaj mari, consecință a antrenării pe durate limitate, cu pauze între versiuni. Un model antrenat în 2023 nu cunoaște evenimente ulterioare și nu poate fi corectat fără reantrenare, un proces costisitor. Recuperarea externă rezolvă această problemă printr-un mecanism mult mai ieftin: actualizarea bazei de cunoștințe, nu a modelului.

Lucrarea de față se concentrează pe aplicarea RAG la extragerea relațiilor între entități, o sarcină de bază în procesarea limbajului natural care constă în identificarea legăturilor dintre entitățile menționate într-un text. Spre deosebire de clasificatoarele specializate, antrenate pentru un set fix de relații, un sistem RAG poate recupera contextul relevant pentru o pereche de entități și poate generaliza către relații noi fără reantrenare, ceea ce îl face o direcție atractivă pentru această sarcină.

Gradul de noutate

Sistemele RAG au apărut oficial în 2020, prin lucrarea „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” publicată de Lewis et al. de la Facebook AI Research (astăzi Meta AI), în contextul exploziei modelelor transformer precum BERT și GPT, care generau răspunsuri creative, dar adesea inexacte sau „halucinate” din cauza limitărilor cunoștințelor lor statice.

De la începutul în 2020 cu RAG „naive”, sistemul a evoluat rapid: în 2023-2024 au apărut variante precum Advanced RAG și Modular RAG; în 2024, Microsoft a introdus GraphRAG pentru relații complexe via knowledge graphs, iar în 2025 s-au impus hibridări (vectori denși și rari, SQL-RAG pentru operații matematice) și Multi-RAG pentru scalabilitate în întreprinderile mari. Până în 2026, RAG este omniprezent în aplicații de business, chatbots și agenți AI, fiind o tehnologie utilizată în producție, trecută de ani de stadiul de prototip.

Obiectivele proiectului

Lucrarea are patru obiective. Primul este prezentarea cadrului teoretic al sistemelor RAG: definiția, arhitectura generală, principalele metode din literatură și metricile de evaluare. Al doilea este implementarea unui demonstrator care compară o arhitectură de bază cu una optimizată, izolând contribuția etapelor de recuperare. Al treilea este evaluarea cantitativă și calitativă a celor două sisteme pe un set de date public, cu teste de semnificație statistică pentru a valida diferențele observate. Al patrulea este identificarea constrângerilor și a direcțiilor

de dezvoltare ulterioară, în special pentru aplicarea la extragerea relațiilor între entități.

Contribuții și rezultate

Contribuția principală este un demonstrator open-source care compară un sistem RAG de tip „naive” cu unul de tip „advanced” pe același set de date și același model lingvistic, însoțit de o interfață grafică pentru testarea interactivă a celor două pipeline-uri. Compararea folosește metrice consacrate în literatura de specialitate (Context Recall, MRR, Exact Match, F1), iar diferențele sunt validate prin teste statistice pereche.

Evaluarea inițială se realizează pe SQuAD, un set de date de referință pentru regăsire și citire extractivă. Această alegere este motivată de structura setului: răspunsul este un fragment de text cu sursă cunoscută, ceea ce permite o măsurare automată și reproductibilă a calității recuperării, izolând contribuția pipeline-ului de recuperare fără a depinde de un model-judecător. Pornind de la această validare, aceeași arhitectură urmează să fie evaluată pe extragerea relațiilor între entități, scopul final al lucrării.

Conținutul lucrării

Lucrarea este structurată în două capitole principale:

Capitolul 1 - Studiu de literatură parcurge domeniul RAG din perspective teoretice și practice. Acesta cuprinde: (1) introducerea și contextul LLM-urilor în extragerea informației; (2) taxonomia sistemelor RAG clasificate după tipul recuperării, integrării și surselor de informație; (3) arhitectura generală a unui sistem RAG cu descrierea componentelor principale; (4) prezentarea metodelor și modelelor reprezentative din literatură (Self-RAG, CRAG, HyDE, GraphRAG); (5) definirea și analiza metricilor de evaluare pentru fiecare etapă a pipeline-ului de RAG (recuperare, generare, augmentare); (6) prezentarea seturilor de date utilizate în evaluarea RAG (T-REx, CRAG, WebNLG, HotpotQA); și (7) discutarea limitărilor și provocărilor actuale ale sistemelor RAG.

Capitolul 2 - Rezultate experimentale prezintă implementarea și comparația empirică a două tipuri de sisteme RAG: (1) Naive RAG - versiunea de bază cu recuperare pe bază de vectori denși și generare de către modelul lingvistic mare; și (2) Advanced RAG - cu recuperare de tip hibrid, reordonare și rescriere a interogărilor LLM-ului. Arhitectura a fost validată mai întâi pe setul de date SQuAD (Stanford Question Answering Dataset), un benchmark extractiv care permite o evaluare automată și precisă a recuperării; evaluarea pe extragerea relațiilor între entități, pe seturi de tip T-REx (relații entitate-entitate din Wikipedia aliniate cu Wikidata), este etapa imediat următoare a lucrării. Capitolul cuprinde atât analiza cantitativă (Context Recall@K, MRR, Exact Match, F1) cât și analiza calitativă a sistemelor RAG; evaluarea cu metrice RAGAS (fidelitate și relevanța răspunsului) este prevăzută ca extensie. Sistemele ilustrate anterior pot fi testate interactiv într-o interfață implementată în Streamlit.

Capitolul 1

Studiu de literatură

1.1 Prezentarea domeniului lucrării de diplomă

1.1.1 Introducere

Revoluția modelelor bazate pe arhitectura transformer, apărută în 2017 [1] și a modelelor lingvistice mari (Large Language Models - LLMs) a dus la o schimbare paradigmatică a abordării problemelor de procesare de limbaj natural. Modele precum BERT și GPT-3 au demonstrat capacități remarcabile în înțelegerea și generarea de text coerent, favorizând o adoptare rapidă a LLM-urilor la nivel global, cercetarea despre LLM-uri devenind un punct central de interes. În Figura 1.1 se poate observa creșterea exponențială a numărului de lucrări despre LLM-uri publicate pe arXiv în perioada 2017–2025. Totuși, LLM-urile de sine stătătoare au o limitare fundamentală, cunoștințele lor sunt fixe, limitate la momentul antrenării, neputând fi actualizate fără a fi reantrenate [2].

1.1.2 De la rețele neuronale la modele lingvistice mari

Pentru a înțelege de ce a apărut RAG, este util un scurt parcurs al evoluției modelelor de procesare a limbajului. Învățarea profundă (eng. *deep learning*) a schimbat radical modul de abordare a sarcinilor de inteligență artificială, înlocuind trăsăturile construite manual cu reprezentări învățate automat din date [3, 4]. În procesarea limbajului natural, acest lucru a însemnat trecerea de la modele care se bazau pe reguli și pe trăsături lexicale la modele care învață singure reprezentări ale cuvintelor și ale propozițiilor.

Saltul decisiv a venit cu arhitectura transformer [1], care a introdus mecanismul de atenție ca element central. Spre deosebire de arhitecturile anterioare, transformerul procesează toate cuvintele unei secvențe în paralel și învață relațiile dintre ele indiferent de distanță, ceea ce l-a făcut potrivit pentru antrenarea pe corpusuri foarte mari. Pe această bază au fost construite modelele lingvistice mari, antrenate pe cantități uriașe de text pentru a prezice cuvântul următor.

Pe măsură ce dimensiunea modelelor și a datelor de antrenare a crescut, capacitățile lor au crescut și ele, dar a devenit tot mai clară o limitare structurală: cunoștințele rămân înghețate la momentul antrenării. Acest lucru a motivat căutarea unor metode prin care un model să poată accesa informație externă, actualizată, la momentul interogării, și a deschis drumul către generarea augmentată prin recuperare.

1.1.3 Problema: Cunoștințe statice care duc la halucinații

Fenomenul de „halucinație” reprezintă un răspuns aparent bine formulat, dar factual incorect, al unui LLM. Modelele generează text pe baza distribuției de probabilități a datelor cu care au fost antrenate, nu pe baza unor cunoștințe verificate. Pentru utilizatorul obișnuit, halucinațiile nu reprezintă un impediment major, însă există aplicații în care acestea sunt inacceptabile (medical, legal, financiar).

Evoluția Exponențială a Cercetării în Domeniul LLM (2017 - 2025)
Sursă Date: API arXiv (Interogare Automată)

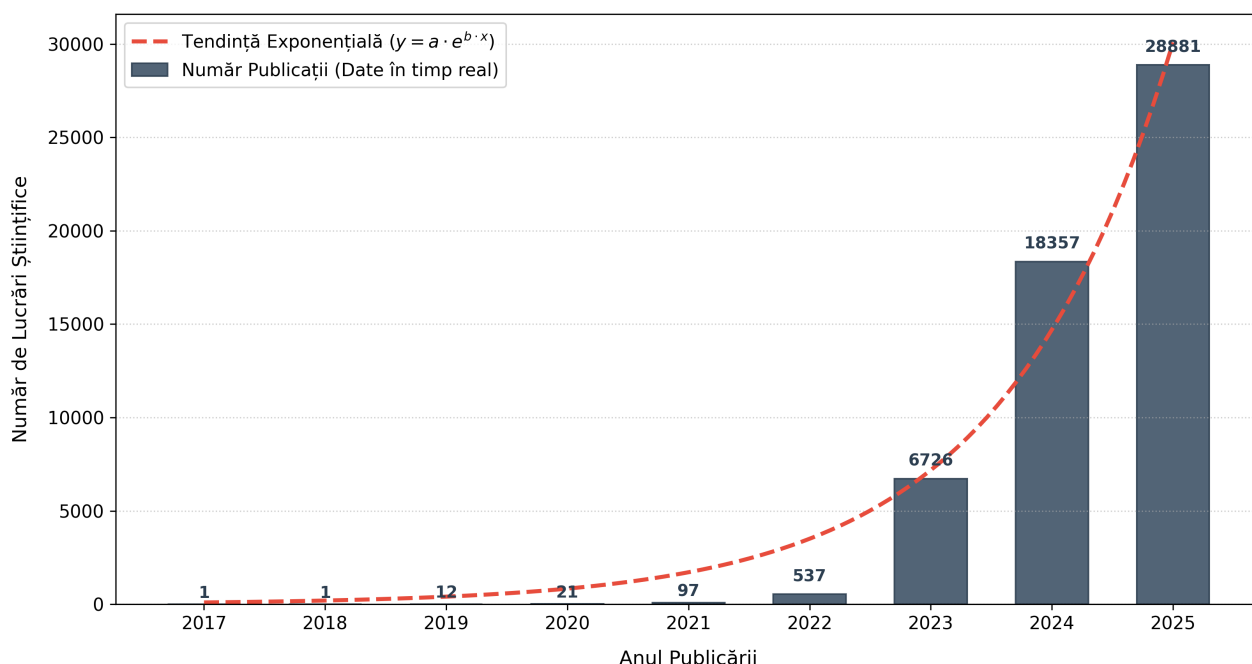


Figura 1.1: Creșterea numărului de lucrări despre lingvistice mari (LLMs) publicate pe arXiv, 2017–2025.

În plus, informația încorporată în parametrii modelului este de natură „statică”, reflectă cunoștințele cunoscute la momentul antrenării. După antrenare, singura metodă de actualizare a informației este procesul costisitor de fine-tuning.

1.1.4 Soluția: Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) este un tip de arhitectură de sistem de inteligență artificială care combină recuperarea informației relevante din surse externe cu capacitatea de generarea a LLM-urilor. În contextul unui sistem RAG, documentele relevante sunt recuperate dintr-o sursă externă de date, încorporate în contextul LLM-ului, iar în final, se generează un răspuns. Aceasta adresează ambele probleme simultan:

1. Furnizează informația necesară unui context factual corect, reducând astfel halucinațiile;
2. Permite modificarea cunoștințelor și extensia contextului fără fine-tuning sau reantrenare.

Termenul „RAG” a fost introdus în 2020 de către Patrick Lewis și colaboratorii săi de la Facebook AI Research (astăzi Meta AI) în lucrarea „*Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*” [5].

Ideea de bază a RAG este simplă, dar puternică: separă cunoștințele de capacitatea de raționament. Modelul lingvistic rămâne responsabil de înțelegerea limbajului și de formularea răspunsului, în timp ce cunoștințele factuale sunt mutate într-o sursă externă, ușor de inspectat și de actualizat. Această separare aduce și un beneficiu de transparentă: fiecare răspuns poate fi însoțit de fragmentele pe care s-a bazat, ceea ce permite utilizatorului să verifice sursa. Spre deosebire de un model care răspunde dintr-o cunoaștere opacă, un sistem RAG face vizibil pe ce se sprijină răspunsul, un aspect important în aplicațiile unde corectitudinea trebuie justificată.

1.1.5 RAG versus fine-tuning

Pentru a adapta un model de limbaj la cunoștințe noi există două abordări principale: fine-tuning-ul și RAG. Fine-tuning-ul continuă antrenarea modelului pe date suplimentare, încorporând noile cunoștințe direct în parametri. RAG lasă parametrii neschimbați și furnizează cunoștințele la momentul interogării, prin recuperare externă.

Cele două abordări rezolvă probleme diferite. Fine-tuning-ul este potrivit pentru a învăța modelul un stil, un format de răspuns sau un domeniu de specialitate, deoarece modifică modul în care modelul generează text. RAG este potrivit pentru a furniza fapte care se schimbă în timp sau care sunt prea numeroase pentru a fi memorate, deoarece baza de cunoștințe poate fi actualizată fără a atinge modelul [2]. Pentru extragerea relațiilor între entități, unde faptele relevante sunt numeroase și pot fi actualizate frecvent, RAG oferă un avantaj clar: adăugarea unui document nou în baza de cunoștințe face imediat disponibile relațiile pe care le conține, fără reantrenare.

Cele două abordări nu se exclud. Un sistem poate folosi un model fine-tuned pe domeniul țintă, alimentat în același timp cu dovezi recuperate prin RAG. În practică, RAG este adesea preferat ca primă soluție, deoarece costul actualizării bazei de cunoștințe este mult mai mic decât costul reantrenării modelului.

1.1.6 Formularea problemei: Extragerea relațiilor între entități

Extragerea relațiilor între entități este o sarcină de bază în procesarea limbajului natural. Dat fiind un text, scopul este de a identifica entitățile menționate și relațiile dintre ele. De exemplu, din textul „Albert Einstein s-a născut în Ulm în 1879”, sistemul trebuie să extragă relațiile (Albert Einstein, născut în, Ulm) și (Albert Einstein, născut în an, 1879).

Formal, fie q o interogare și $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ o mulțime de documente. Un sistem RAG acționează în două etape. Prima etapă: un model de recuperare $p_\eta(\cdot | q)$ alege o submulțime $\mathcal{Z} \subseteq \mathcal{D}$ de documente-candidat. A doua etapă: un model lingvistic mare generativ p_θ generează răspunsul y condiționat de interogare dar și de documentele recuperate:

$$p(y | q) = \sum_{z \in \mathcal{Z}} p_\eta(z | q) p_\theta(y | q, z). \quad (1.1)$$

RAG aduce o alternativă rețelelor neuronale de clasificare create pentru sarcini specifice. Sistemul recuperează pasaje relevante pentru fiecare pereche de entități, iar LLM-ul extrage relația pentru contextul recuperat, astfel generalizând pentru relații noi, fără reantrenare [2].

1.1.7 Relevanță și aplicații

Sistemele RAG sunt relevante într-o gamă largă de domenii unde accesul dinamic la informații externe este esențial pentru generarea de răspunsuri de calitate [2].

- **Construirea grafurilor de cunoștințe:** Wikipedia, DBpedia, Freebase și alte baze de date semantice structurate sunt construite parțial prin extragerea automată a relațiilor
- **Asistenți inteligenți:** Chatbots și agenți de inteligență artificială care necesită cunoștințe actualizate constant
- **Domenii specializate:** Bioinformatică (extragerea relațiilor proteină-proteină), drept (identificarea precedentilor), medicină (relații medicament-boală)

- **Indexare și căutare:** Motoarele de căutare moderne folosesc RAG pentru a livra răspunsuri directe

Firul comun al acestor aplicații este nevoia de răspunsuri fundamentate pe surse verificabile, nu pe cunoștințele interne ale modelului. În extragerea relațiilor, aceasta înseamnă că fiecare relație extrasă poate fi urmărită până la fragmentul de text care o susține, ceea ce face sistemul auditabil. Pentru domenii precum medicina sau dreptul, unde o relație greșit extrasă poate avea consecințe serioase, această trasabilitate este la fel de importantă ca acuratețea în sine.

1.2 Taxonomie

1.2.1 Motivația definirii unei taxonomii

Domeniul RAG s-a dezvoltat rapid, producând numeroase variante și îmbunătățiri ale arhitecturii originale. Identificarea similarităților și diferențelor între abordări este dificilă fără o clasificare sistematică. Lucrările Gao et al. (2024) [2] și Peng et al. (2025) [6] au fost folosite ca referință pentru această secțiune, care introduce o taxonomie bazată pe aspectele tehnologice cheie. Trebuie reținut că taxonomia este limitată la specificul acestei lucrări (RAG pentru extragerea relațiilor) și nu acoperă întreg domeniul.

O taxonomie are două roluri practice. Primul este de a oferi un vocabular comun: atunci când o metodă este descrisă ca „recuperare hibridă cu reranking post-recuperare”, criteriile taxonomiei spun imediat unde se plasează în spațiul soluțiilor. Al doilea este de a ghida deciziile de proiectare: alegerea unei arhitecturi pentru o aplicație concretă revine la alegerea unei opțiuni pe fiecare criteriu, în funcție de tipul datelor, de latența acceptabilă și de natura sarcinii. Pentru această lucrare, taxonomia justifică direct cele două arhitecturi implementate și comparate în demonstrator, Naive și Advanced RAG.

1.2.2 Criterii de clasificare

1. Tipul mecanismului de recuperare

Primul criteriu privește modul în care sistemul găsește documentele relevante. Distincția fundamentală este între potrivirea pe cuvinte (sparse) și potrivirea pe sens (dense), iar combinarea lor (hybrid) urmărește să exploateze avantajele ambelor.

- **Sparse retrieval:** Metode tradiționale de recuperare de IR (Information Retrieval) precum BM25, care se bazează pe potrivirea cuvintelor cheie. Rapid dar sensibil la variații lexicale și sinonime.
- **Dense retrieval:** Folosește reprezentări numerice a textului din documente (și interogarea către LLM) în spații vectoriale continue. Poate captura nuanțele semantice, dar costul computațional este mai ridicat.
- **Hybrid retrieval:** Combină sparse și dense prin fusion (ex. Reciprocal Rank Fusion), exploatând avantajele ambelor abordări.

2. Modul de integrare între recuperare și generare

Al doilea criteriu privește cât de strâns sunt legate cele două componente. Sistemele decuplate sunt mai ușor de construit și de înlocuit pe bucăți, în timp ce sistemele integrate pot fi optimizate împreună, dar sunt mai greu de modificat.

- **Modular:** Componentele sunt decuplate. Recuperatorul și generatorul sunt independente și pot fi optimizate separat, oferind flexibilitate maximă.

- **End-to-end:** Recuperarea și generarea sunt cuplate, și sunt optimizate laolaltă, de regulă prin antrenare (gradient descent).

3. Tipul sursei de cunoștințe

Al treilea criteriu privește forma datelor din care se recuperează. Sursele nestructurate sunt abundente și ușor de adunat, dar necesită indexare semantică, iar sursele structurate modelează explicit relațiile, ceea ce le face foarte potrivite pentru extragerea relațiilor între entități.

- **Nestructurată:** Documente text din diverse surse precum site-uri web (ex. articole Wikipedia), PDF-uri, documente Word. Pentru a putea fi procesate, necesită indexare și analiză semantică.
- **Structurată:** Grafuri de cunoștințe cu entități și relații explicit modelate (DBpedia, Wikidata, Freebase), permitând navigare relațională.
- **Hibridă:** Combinație ale amândurora, exploatând avantaje complementare.

4. Etapa augmentării

Al patrulea criteriu privește momentul în care se intervine asupra procesului pentru a-i îmbunătăți calitatea. Optimizările pot fi aplicate înainte, în timpul sau după recuperare, iar un sistem avansat le combină de obicei pe toate trei.

- **Pre-retrieval:** Optimizarea pre-recuperare prin extinderea și/sau reformularea întrebării și descompunere semantică.
- **At-retrieval:** Optimizarea la momentul recuperării, prin selectarea parametrilor k , strategii adaptive de recuperare.
- **Post-retrieval:** Optimizare după recuperare, prin reranking, compactarea contextului și filtrare.

1.2.3 Categoriile principale

Pe baza criteriilor de mai sus, literatura distinge patru categorii principale de arhitecturi RAG, ordonate aici de la cea mai simplă la cea mai complexă. Ele nu sunt etape obligatorii, ci puncte pe un spectru: un sistem real poate combina elemente din mai multe categorii.

Naive RAG

Paradigma originală din lucrarea Lewis et al. 2020 [5]: indexare, urmată de recuperare de tip dense și generare. Recuperatorul și generatorul sunt elemente separate, și nu se folosesc optimizări. Este arhitectura de bază, folosită ca referință în majoritatea studiilor [5], incluzând această teză.

Advanced RAG

Adaugă optimizări pre- și post-recuperare, precum: rescriere a întrebărilor, recuperare hibridă, și reranking cu model special antrenat. Dovedește îmbunătățiri semnificative, între 5 și 15% pe benchmark-uri, cu costuri computaționale moderate. Spre deosebire de Modular RAG, păstrează structura liniară a fluxului Naive, dar inserează etape de optimizare înainte și după recuperare. Acest echilibru între simplitate și performanță îl face o alegere practică pentru multe aplicații: aduce o parte importantă din beneficiile arhitecturilor complexe, fără complexitatea lor de implementare. Lucrarea de față implementează acest tip de arhitectură.

Modular RAG

Diferă de metodele Naive și Advanced în structură, fiecare element din arhitectura Modular RAG este un modul separat (ex. recuperare, memorare, rutare, predicție, căutare). Modulele

pot fi rearanjate, înlocuite sau apelate în bucle, ceea ce permite fluxuri de lucru complexe, precum recuperarea iterativă sau rutarea interogării către surse diferite în funcție de tip. Este cea mai adaptabilă arhitectură, însă introduce și cel mai înalt nivel de complexitate, atât la implementare, cât și la depanare.

Graph RAG

Specifică pentru surse structurate de date: indexare pe grafuri de cunoștințe, recuperare utilizând relații semantice, generare pe bază de subgrafuri. În loc să caute fragmente de text izolate, Graph RAG navighează legăturile dintre entități, ceea ce îi permite să răspundă la întrebări care presupun coroborarea mai multor relații. Este ideal pentru extragerea relațiilor din grafuri de cunoștințe, dar necesită structurarea prealabilă a datelor sub formă de graf, un pas costisitor. Această categorie este descrisă în detaliu de Peng et al. (2025) [6].

Figura 1.2 sintetizează taxonomia adoptată sub forma unei hărți conceptuale, organizată pe cele patru criterii de clasificare descrise mai sus. Cele patru categorii principale (Naive, Advanced, Modular și Graph RAG) se diferențiază prin combinațiile de opțiuni alese pe fiecare criteriu. Demonstratorul din această lucrare implementează două dintre aceste categorii, Naive RAG (recuperare densă, fără optimizări) și Advanced RAG (recuperare hibridă, optimizări pre- și post-recuperare), și le compară direct pentru a măsura efectul optimizărilor de recuperare.

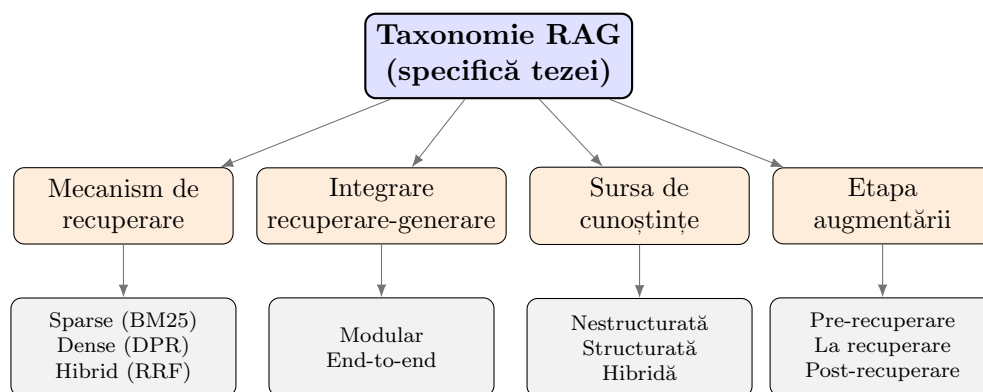


Figura 1.2: Harta conceptuală a taxonomiei adoptate, structurată pe cele patru criterii relevante pentru extragerea relațiilor între entități [2, 6].

1.3 Arhitectura generală a unui sistem de tip RAG

Un sistem RAG funcționează în două faze: o fază offline de indexare și o fază online de inferență. Componentele principale sunt interogarea utilizatorului, baza de cunoștințe, mecanismul de recuperare și modelul generativ [2].

1.3.1 Componentele sistemului

Interogarea reprezintă nevoia de informație a utilizatorului și este punctul de pornire al fluxului online. Rolul ei este dublu: pe de o parte declanșează recuperarea, pe de altă parte stabilește criteriul față de care se măsoară relevanța fragmentelor. În formele de bază, textul interogării este transmis direct mecanismului de recuperare, fără prelucrare.

Sistemele avansate aplică mai întâi transformări ale interogării: reformulare semantică, extindere cu termeni sinonimi sau descompunere în sub-interogări, pentru a crește acoperirea și a reduce sensibilitatea față de formulări ambigue [2]. Pentru extragerea relațiilor, formularea interogării contează mult, deoarece aceeași relație poate fi exprimată în multe feluri, iar o singură formulare poate rata documentele care folosesc alți termeni.

Baza de cunoștințe este sursa externă din care se extrage informația și determină, prin conținutul ei, ce poate și ce nu poate ști sistemul. Spre deosebire de cunoștințele parametrice ale modelului, baza de cunoștințe poate fi actualizată oricând, fără reantrenare. Documentele trec printr-o etapă de indexare offline: sunt împărțite în fragmente (eng. *chunks*), codificate cu un model de reprezentare vectorială bazat pe arhitectura transformer [1] și stocate într-o bază de date vectorială [7].

Strategia de fragmentare influențează direct calitatea recuperării: fragmentele prea scurte pierd contextul necesar pentru identificarea relațiilor între entități, iar cele prea lungi diluează semnalul relevant. Calitatea bazei de cunoștințe limitează performanța întregului sistem, deoarece o relație care nu apare în niciun document nu poate fi recuperată, oricât de bun ar fi mecanismul de recuperare.

Mecanismul de recuperare punctează fragmentele din baza vectorială în raport cu interogarea și returnează primele k rezultate. Rolul lui este de a aduce în fața modelului generativ exact dovezile necesare, nici prea puține, încât să rateze informația, nici prea multe, încât să o îngroape în zgomot. Sistemele dense calculează similaritatea cosinus între vectorul interogării și vectorii fragmentelor [7], captând asemănarea de sens. Sistemele sparse aplică potrivire ponderată de termeni, BM25 reprezentând standardul din domeniu [8], captând potrivirea lexicală exactă.

Sistemele hibride combină scorurile dense și sparse prin Reciprocal Rank Fusion (RRF) [9], pentru a beneficia de ambele tipuri de potrivire. Opțional, un model de reranking de tip cross-encoder rafinează ordinea inițială, reevaluând fiecare pereche interogare-fragment cu o precizie mai mare decât similaritatea vectorială. Acest pas suplimentar este foarte util pentru extragerea relațiilor, unde fragmentul corect trebuie să conțină simultan ambele entități și formularea relației.

Modelul generativ primește contextul augmentat, format din interogarea originală și fragmentele recuperate, și produce răspunsul final [5]. Rolul lui nu este de a ști răspunsul, ci de a-l sintetiza din dovezile furnizate: identifică în fragmente informația relevantă, o leagă de întrebare și o exprimă coerent. În cazul extragerii relațiilor, modelul localizează în context legătura dintre entități și o formulează ca răspuns.

Arhitecturile orientate spre fidelitate constrâng explicit modelul să fundamenteze răspunsurile pe dovezile recuperate, reducând astfel rata de halucinații [10]. Constrângerea se impune de obicei prin promptul de sistem, care instruește modelul să răspundă doar din context și să semnaleze explicit absența informației, în loc să completeze din cunoștințele sale interne.

1.3.2 Fluxul de date

Figura 1.3 ilustrează cele două trasee ale fluxului de date. Offline, documentele sunt procesate o singură dată: fragmentate, codificate și stocate în baza vectorială. Online, fiecare interogare parcurge procesarea interogării, recuperarea fragmentelor, augmentarea contextului și generarea răspunsului.

Separarea în două faze are o motivație practică. Indexarea este o operație costisitoare, deoarece presupune codificarea întregului corpus, dar trebuie făcută o singură dată și poate fi refolosită pentru orice număr de interogări. Inferența, în schimb, trebuie să fie rapidă, deoarece se execută la fiecare cerere a utilizatorului. De aceea merită să optimizăm indexarea offline, chiar dacă durează mult, pentru ca recuperarea online să fie cât mai rapidă. Atunci când corpusul se schimbă, doar documentele noi trebuie codificate și adăugate, fără a reface întregul index.

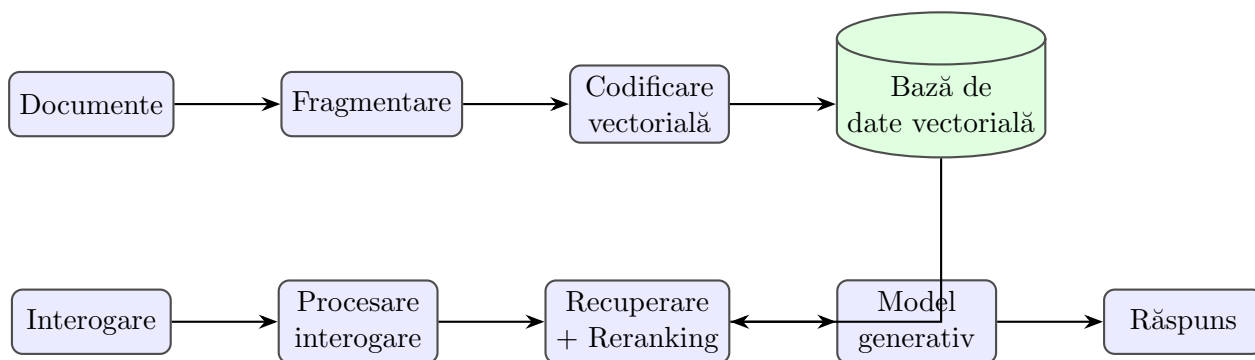


Figura 1.3: Arhitectura generală a unui sistem RAG: indexare offline (sus) și inferență online (jos) [2, 7].

1.3.3 Exemplu de flux pentru extragerea relațiilor

Un exemplu concret clarifică rolul fiecărei componente. Să presupunem interogarea „Unde s-a născut Marie Curie?”, pentru care răspunsul corect este tripletul (Marie Curie, loc de naștere, Varșovia).

În faza online, interogarea este mai întâi codificată într-un vector. Mecanismul de recuperare caută în baza vectorială fragmentele cele mai apropiate semantic și returnează, de exemplu, un pasaj din articolul Wikipedia despre Marie Curie care conține propoziția „Maria Skłodowska s-a născut la Varșovia în 1867”. Acest fragment este introdus în contextul modelului generativ împreună cu interogarea originală. Modelul generativ identifică în fragment legătura dintre entitatea-subiect și locul de naștere și produce răspunsul fundamentat pe dovada recuperată, nu pe cunoștințele sale interne.

Acest exemplu evidențiază două aspecte importante pentru extragerea relațiilor. Primul: recuperarea trebuie să aducă un fragment care conține simultan entitatea și indiciul lexical al relației, altfel modelul nu are pe ce să se bazeze. Al doilea: variația lexicală (interogarea spune „Marie Curie”, iar textul „Maria Skłodowska”) poate face recuperarea densă să rateze fragmentul corect, problemă pe care optimizările din Advanced RAG, precum reformularea interogării și recuperarea hibridă, o atenuează.

1.3.4 Strategii de fragmentare

Fragmentarea (eng. *chunking*) este pasul prin care documentele sunt împărțite în unități inde-xabile, iar alegerea strategiei influențează direct calitatea recuperării. Există câteva abordări uzuale.

Fragmentarea de lungime fixă împarte textul în bucăți de un număr fix de tokenuri sau caractere, eventual cu o suprapunere între fragmente consecutive pentru a nu pierde contextul de la graniță. Este simplă și rapidă, dar poate tăia o propoziție sau o relație în două. Fragmentarea pe granițe naturale, la nivel de propoziție sau paragraf, respectă structura textului și păstrează relațiile intacte în interiorul unui fragment, dar produce fragmente de lungimi inegale. Fragmentarea semantică grupează propozițiile înrudite pe baza similarității de repre-zentare, încercând să mențină împreună informația coerentă tematic, dar cu un cost de calcul mai mare la indexare.

Compromisul fundamental este între specificitate și context [2]. Fragmentele scurte sunt specifice, dar pot pierde contextul necesar pentru a identifica o relație care se întinde pe mai multe propoziții. Fragmentele lungi păstrează contextul, dar diluează semnalul relevant și introduc zgomot în promptul modelului generativ. Pentru extragerea relațiilor, fragmentul ideal conține ambele entități și formularea relației dintre ele, ceea ce favorizează fragmentarea

pe granițe naturale, la nivel de paragraf. Demonstratorul din această lucrare folosește paragraful ca unitate de fragmentare.

1.3.5 Modele de embedding și baze de date vectoriale

Calitatea recuperării dense depinde de modelul de embedding, care transformă textul într-un vector numeric. Modelele de embedding moderne sunt encodere bazate pe arhitectura transformer [1], antrenate astfel încât textele apropiate ca sens să aibă vectori apropiați în spațiul de reprezentare. Alegerea modelului implică un compromis între calitate și viteză: modelele mari produc reprezentări mai bune, dar sunt lente și costisitoare, în timp ce modelele compacte precum familia MiniLM oferă un raport bun între acuratețe și cost, motiv pentru care sunt des folosite în sistemele RAG fără constrângeri hardware mari.

Vectorii rezultați sunt stocați într-o bază de date vectorială, care permite căutarea rapidă a celor mai apropiați vecini. Căutarea exactă a vecinilor într-un corpus mare ar fi prea lentă, așa că aceste baze de date folosesc algoritmi de căutare aproximativă a vecinilor (eng. *approximate nearest neighbor*), de tipul grafurilor HNSW, care găsesc rapid vectori apropiați cu o pierdere mică de acuratețe. Soluții uzuale sunt FAISS, ChromaDB și Milvus. Demonstratorul din această lucrare folosește ChromaDB, configurat cu similaritate cosinus, deoarece oferă persistență locală simplă și se integrează direct cu modelele de embedding folosite.

1.3.6 Construcția promptului și augmentarea contextului

După recuperare, fragmentele selectate trebuie integrate în promptul modelului generativ. Acest pas, deși adesea trecut cu vederea, influențează direct calitatea răspunsului. Un prompt RAG tipic are trei părți: o instrucțiune de sistem care stabilește comportamentul modelului, contextul format din fragmentele recuperate și interogarea utilizatorului.

Instrucțiunea de sistem este locul în care se impune constrângerea de fundamentare (eng. *grounding*): modelul este instruit să răspundă exclusiv pe baza contextului furnizat și să semnaleze explicit când informația lipsește, în loc să completeze din cunoștințele sale interne. Această constrângere reduce halucinațiile și este caracteristică sistemelor orientate spre fidelitate [10]. Ordinea fragmentelor în context contează și ea: modelele tind să acorde mai multă atenție începutului și sfârșitului unui context lung, fenomen care motivează plasarea fragmentelor cele mai relevante pe pozițiile de capăt [11].

Pentru extragerea relațiilor, modul de construire a promptului determină dacă modelul are simultan în față ambele entități și formularea relației. Un context bine construit, în care fragmentele relevante nu sunt diluate de pasaje de umplutură, crește șansa ca relația corectă să fie identificată și exprimată în răspuns.

1.3.7 Compromisuri în proiectarea unui sistem RAG

Proiectarea unui sistem RAG presupune o serie de decizii între care există tensiuni. Niciuna nu are un răspuns universal corect; alegerea depinde de aplicație, de date și de resursele disponibile. Înțelegerea acestor compromisuri ajută la interpretarea rezultatelor experimentale din capitolul următor.

Primul compromis privește dimensiunea fragmentelor. Fragmentele mici sunt specifice și aduc puțin zgomot, dar riscă să piardă contextul necesar pentru identificarea unei relații care se întinde pe mai multe propoziții. Fragmentele mari păstrează contextul, dar diluează semnalul și consumă mai mult din spațiul limitat al promptului. Nu există o dimensiune optimă universală; ea depinde de structura textului și de tipul întrebărilor.

Al doilea compromis privește numărul de fragmente recuperate, parametrul k . Un k mic riscă să rateze informația relevantă, în timp ce un k mare introduce pasaje irelevante care pot induce în eroare modelul generativ [11]. Creșterea lui k nu îmbunătățește monoton performanța, ci atinge un punct dincolo de care zgomotul depășește beneficiul acoperirii.

Al treilea compromis privește alegerea între recuperarea densă și cea rară. Recuperarea densă captează sensul, dar ratează termenii rari; recuperarea rară captează potrivirile lexicale exacte, dar nu înțelege sinonimele. Recuperarea hibridă încearcă să le combine, dar cu o complexitate și un cost de calcul mai mari.

Al patrulea compromis, poate cel mai vizibil în practică, este între calitate și latență. Fiecare etapă de optimizare, de la expansiunea interogărilor la reranking, îmbunătățește calitatea recuperării, dar adaugă timp de procesare. În aplicațiile interactive, unde utilizatorul așteaptă un răspuns, acest compromis devine o constrângere de proiectare, nu doar o opțiune. Partea experimentală a lucrării măsoară direct acest compromis pe cele două arhitecturi comparate.

1.4 Metode și modele RAG

1.4.1 Naive RAG

Formularea originală a RAG, propusă de Lewis et al. (2020) [5], combină un recuperator bazat pe Dense Passage Retrieval (DPR) cu un model generativ de tip sequence-to-sequence, optimizate parțial în comun. Fluxul de lucru este liniar: indexare, recuperare densă, generare. Această variantă servește ca referință în majoritatea studiilor din literatura de specialitate [2].

Avantajul principal al Naive RAG este simplitatea: are puține componente, este ușor de implementat și de înțeles, iar costul computațional este redus, deoarece o singură interogare densă este suficientă. Tocmai de aceea servește ca referință în majoritatea evaluărilor comparative, inclusiv în cadrul acestei teze.

Dezavantajele sunt bine documentate. Sistemul nu gestionează variațiile lexicale (sinonime, parafrazări), deoarece se bazează exclusiv pe similaritatea semantică, nu evaluează calitatea documentelor recuperate și nu filtrează fragmentele irelevante înainte de generare [2]. Pe interogări cu termeni rari sau numerici, recuperarea densă ratează frecvent fragmentul corect, ceea ce limitează direct calitatea răspunsului final.

1.4.2 Advanced RAG

Advanced RAG adaugă optimizări pre-recuperare și post-recuperare față de varianta de bază [2]. Pre-recuperare: fragmentare semantică, îmbogățirea fragmentelor cu metadate, generare de interogări multiple. Post-recuperare: reranking cu model cross-encoder, compactarea contextului pentru a reduce zgomotul din prompt. Scorurile dense și sparse sunt combinate prin RRF [9]. Studiile raportează îmbunătățiri de 5–15% față de Naive RAG pe benchmark-uri standard.

Avantajul Advanced RAG este creșterea calității recuperării: recuperarea hibridă prinde atât potrivirile semantice, cât și pe cele lexicale, iar rerankingul așază fragmentele cu adevărat relevante pe primele poziții. Dezavantajul este costul: fiecare optimizare adaugă latență, iar expansiunea interogărilor presupune un apel suplimentar la un model de limbaj, ceea ce poate crește timpul de răspuns de câteva ori față de varianta de bază, după cum arată și partea experimentală a acestei lucrări.

Pentru extragerea relațiilor, optimizările post-recuperare sunt direct relevante: identificarea corectă a unei relații necesită fragmente care conțin simultan ambele mențiuni de entitate și indiciile lexicali ai relației. Reranking-ul crește șansa ca aceste fragmente să ajungă în contextul

modelului generativ. Aceasta este arhitectura pe care lucrarea de față o implementează și evaluează în partea experimentală.

1.4.3 Recuperarea densă: DPR și ColBERT

Recuperarea densă stă la baza majorității sistemelor RAG moderne, inclusiv a celui implementat în această lucrare. Dense Passage Retrieval (DPR) [7] folosește două encodere separate, unul pentru interogare și unul pentru pasaje, antrenate astfel încât perechile relevante să aibă vectori apropiați în spațiul de reprezentare. La recuperare, interogarea este codificată o singură dată, iar pasajul cu cea mai mare similaritate cosinus este returnat. Avantajul față de metodele lexicale precum BM25 este captarea sinonimiei și a parafrazării: DPR poate lega „loc de naștere” de „s-a născut la” chiar dacă termenii nu coincid. Limitarea este că un singur vector pentru întregul pasaj poate dilua termenii rari sau specifici.

ColBERT [12] rafinează această idee prin interacțiune târzie (eng. *late interaction*). În loc de un singur vector per pasaj, ColBERT păstrează câte un vector pentru fiecare token și calculează relevanța prin suma similarităților maxime dintre tokenurile interogării și cele ale pasajului. Acest nivel fin de detaliu îmbunătățește precizia, mai ales pe interogări cu termeni specifici, dar crește costul de stocare și de calcul, deoarece indexul trebuie să rețină reprezentări la nivel de token. Pentru extragerea relațiilor, interacțiunea târzie ajută la potrivirea precisă a mențiunilor de entitate, care sunt adesea termeni rari.

1.4.4 Hypothetical Document Embeddings (HyDE)

HyDE [13] folosește un model de limbaj pentru a genera un pseudo-document care ar conține răspunsul la interogare. Acest pseudo-document este codificat și se caută documente reale similare prin recuperare densă. Encoderele contrastive elimină erorile factuale din pseudo-document, păstrând tiparele de relevanță semantică [13].

Avantajul principal al HyDE este că îmbunătățește recuperarea fără a necesita date de antrenare etichetate pentru interogări. Dezavantajul este dependența de calitatea generării inițiale: un pseudo-document slab generat poate conduce recuperarea în direcția greșită. Pentru extragerea relațiilor, HyDE poate ajuta atunci când interogarea este formulată abstract: pseudo-documentul generat conține adesea formulări concrete ale relației căutate, apropiind recuperarea de fragmentele care exprimă explicit acea relație.

1.4.5 Reformularea și expansiunea interogărilor

O categorie de tehnici pre-recuperare vizează direct interogarea, înainte ca aceasta să ajungă la mecanismul de recuperare. Motivul este că o singură formulare a întrebării poate rata documentele relevante exprimate cu alte cuvinte. Reformularea (eng. *query rewriting*) transformă interogarea într-o formă mai potrivită pentru recuperare, de exemplu clarificând un termen ambiguu sau eliminând cuvinte de umplură. Expansiunea cu interogări multiple (eng. *multi-query*) generează mai multe variante ale interogării originale, fiecare căutată independent, iar rezultatele sunt apoi combinate.

Ideea din spatele expansiunii cu interogări multiple este creșterea acoperirii: dacă una dintre reformulări folosește termenii care apar efectiv în document, fragmentul corect este recuperat chiar dacă formularea originală îl rata. Costul este un număr mai mare de recuperări și, de obicei, un apel suplimentar la un model de limbaj pentru a genera reformulările. Pentru extragerea relațiilor, această tehnică este utilă deoarece aceeași relație poate fi exprimată în multe feluri („s-a născut la”, „locul nașterii”, „originar din”), iar mai multe reformulări cresc

șansa de a potrivi formularea din text. Demonstratorul din această lucrare folosește expansiunea cu interogări multiple ca parte a pipeline-ului Advanced RAG.

1.4.6 REALM, Fusion-in-Decoder și RETRO

REALM [14] integrează recuperarea chiar în etapa de pre-antrenare, tratând recuperatorul ca o variabilă latentă optimizată împreună cu modelul de limbaj. Abordarea arată că un recuperator co-antrenat cu generatorul înțelege mai bine ce tipuri de documente sunt utile pentru diferite sarcini.

Fusion-in-Decoder (FiD) [15] codifică fiecare pasaj recuperat independent prin encoder, dar realizează fuziunea la nivelul decoderului. Astfel, modelul poate agrega dovezi din surse multiple fără a sufoca encoderele cu un context prea lung. RETRO [16] extinde această idee la scara corpusurilor de zeci de trilioane de tokenuri, fără a crește proporțional numărul de parametri ai modelului, obținând eficiență parametrică pe seama unei infrastructuri mai complexe.

Pentru extragerea relațiilor, mecanismul de fuziune din FiD contează mult: o relație poate fi susținută de dovezi distribuite în mai multe pasaje, iar fuziunea la nivelul decoderului permite modelului să combine aceste dovezi fără a fi limitat de lungimea unui singur context. Spre deosebire de concatenarea simplă a fragmentelor, această abordare scalează mai bine cu numărul de pasaje recuperate.

Avantajul comun al acestor trei abordări este integrarea strânsă dintre recuperare și model: recuperatorul nu mai este o componentă separată, ci este antrenat împreună cu generatorul, ceea ce îmbunătățește relevanța documentelor aduse. Dezavantajul comun este costul: REALM și RETRO presupun infrastructură de antrenare complexă și corpusuri foarte mari, iar integrarea profundă face aceste sisteme mai greu de modificat sau de înlocuit pe componente decât arhitecturile modulare.

1.4.7 RAPTOR

RAPTOR [17] construiește un arbore de rezumate de jos în sus: fragmentele de text sunt grupate în clustere, rezumate, iar procesul se repetă recursiv, generând niveluri de abstractizare din ce în ce mai înalte. La inferență, recuperarea operează pe acest arbore și poate agrega informații de la mai multe niveluri de abstractizare simultan.

Metoda obține rezultate bune pe sarcini care necesită raționament distribuit pe mai multe documente [17]. Avantajul față de recuperarea standard este că furnizează modelului generativ atât detalii fine, cât și context global. Dezavantajul constă în costul de construire și actualizare a arborelui. Pentru extragerea relațiilor, nivelurile de rezumat ale arborelui pot ajuta la identificarea relațiilor implicite, care nu apar formulat într-o singură propoziție, ci reies din coroborarea mai multor pasaje rezumate la un nivel superior de abstractizare.

1.4.8 Corrective RAG (CRAG)

CRAG [18] adaugă un evaluator al calității documentelor recuperate. Când relevanța fragmentelor este insuficientă, sistemul declanșează acțiuni corective: recuperare suplimentară sau căutare pe web, urmată de rafinarea cunoștințelor prin descompunere și recombinare a informației. Dacă documentele recuperate sunt de bună calitate, fluxul rămâne cel standard.

Mecanismul de auto-corecție face sistemul mai robust față de recuperările greșite și reduce riscul extragerii unei relații dintr-un context irelevant. CRAG este una dintre primele abordări care tratează explicit eșecul recuperării ca un caz de gestionat, nu ca o excepție rară.

Avantajul principal al CRAG este robustețea: sistemul nu se prăbușește atunci când recuperarea inițială eșuează, ci caută alternative. Limitarea este costul suplimentar al evaluatorului

și al recuperărilor corective, care pot dubla timpul de răspuns pe interogările dificile. Pentru extragerea relațiilor, capacitatea de a respinge un context irelevant înainte de generare este foarte valoroasă, deoarece o relație extrasă dintr-un fragment greșit este o eroare greu de detectat ulterior.

1.4.9 Self-RAG

Self-RAG [10] antrenează un singur model să decidă adaptiv când să recupereze, când să genereze și când să critice propriile răspunsuri, folosind tokenuri speciale de reflecție introduse în vocabularul modelului. Modelul evaluează relevanța fragmentului recuperat și gradul de fundamentare factuală a răspunsului generat înainte de a-l returna utilizatorului.

Față de CRAG, care utilizează un evaluator extern, Self-RAG internalizează întregul proces de evaluare și corecție. Fidelitatea și controlabilitatea cresc [10]. Pentru extragerea relațiilor, posibilitatea de a verifica dacă tipul de relație prezis este susținut de dovezile recuperate este direct utilă, deoarece reduce numărul de relații extrase din fragmente slab relevante.

Avantajul Self-RAG este că evaluarea și generarea folosesc același model, fără componente externe de întreținut. Limitarea este că modelul trebuie antrenat special pentru a produce tokenurile de reflecție, ceea ce presupune date de antrenare dedicate și un cost de pregătire mai mare decât în cazul unui pipeline modular. Acest compromis între simplitatea de operare și costul de antrenare este reprezentativ pentru tensiunea dintre arhitecturile integrate și cele modulare discutată în taxonomie.

1.4.10 Adaptive-RAG și FLARE

Adaptive-RAG [19] alege dinamic strategia de recuperare potrivită pentru fiecare interogare, fără recuperare, cu recuperare unică sau cu recuperare iterativă, pe baza complexității estimate cu un clasificator mic antrenat separat. Interogările simple nu necesită recuperare; cele complexe, care implică mai mulți pași de raționament, beneficiază de recuperare iterativă.

FLARE [20] abordează problema diferit, prin recuperare activă: modelul generează propoziția următoare și, atunci când detectează tokenuri cu probabilitate scăzută (indiciu de incertitudine), folosește predicția respectivă ca interogare și recuperează documente suplimentare înainte de a regenera propoziția. Recuperarea iterativă din Adaptive-RAG ajută la identificarea relațiilor distribuite pe mai multe documente, iar FLARE reduce halucinațiile în extragerea pe texte lungi.

Avantajul ambelor metode este eficiența adaptivă: resursele de recuperare sunt cheltuite doar atunci când este nevoie, evitând recuperările inutile pe interogări simple. Dezavantajul este complexitatea suplimentară și, în cazul FLARE, costul recuperării repetate pe parcursul generării, care poate încetini semnificativ producerea unui răspuns lung. În plus, Adaptive-RAG depinde de calitatea clasificatorului de complexitate: o clasificare greșită trimite o interogare dificilă pe ruta fără recuperare, ratând documentele necesare.

1.4.11 GraphRAG și HippoRAG

GraphRAG [21] construiește un graf de cunoștințe din corpus prin extragerea entităților și relațiilor, structurând informația în comunități ierarhice rezumate. Recuperarea operează pe acest graf prin traversarea relațiilor semantice, permițând raționamentul global, nu doar căutarea locală de fragmente similare.

HippoRAG [22] integrează un model de limbaj, un graf de cunoștințe și Personalised PageRank: corpusul este transformat într-un graf fără schemă predefinită, iar conceptele din interogare servesc ca noduri de pornire pentru propagarea pe graf. Ambele abordări se înscriu

în direcția de cercetare Graph RAG descrisă de Peng et al. (2025) [6]. Avantajul comun este descoperirea relațiilor implicite distribuite în mai multe pasaje, ceea ce le face direct relevante pentru sarcina de extragere a relațiilor. Limitarea comună este costul de construire și actualizare a grafului.

1.4.12 RAG pentru contexte lungi și aplicații specializate

Creșterea numărului de pasaje recuperate nu garantează îmbunătățirea performanței. Pasajele irelevante din contexte lungi degradează calitatea generării, un fenomen documentat de Jin et al. (2024) [11]. Problema selectivității devine la fel de importantă ca problema acoperirii.

Problema integrării contextelor lungi este foarte relevantă pentru extragerea relațiilor, unde o relație poate fi susținută de dovezi distribuite în mai multe documente. Pe de o parte, sunt necesare mai multe pasaje pentru a acoperi toate dovezile; pe de altă parte, prea multe pasaje irelevante diluează semnalul. Echilibrul dintre acoperire și selectivitate devine astfel o decizie de proiectare centrală, nu un detaliu de reglaj.

Alte extensii documentate în literatura recentă includ RAG multimodal, care integrează documente text, imagini, audio și video [23], MMed-RAG pentru modele vizuale din domeniul medical [24], RAP pentru personalizarea răspunsurilor în funcție de preferințele utilizatorului [25] și abordări de recuperare din colecții de mii de documente [26]. Aceste direcții arată că RAG nu este o singură arhitectură, ci o familie de tehnici care se adaptează la tipul sursei de date și la cerințele aplicației. Pentru extragerea relațiilor din text, varianta relevantă rămâne RAG pe documente text, cu accent pe recuperarea precisă a fragmentelor care conțin entitățile și relația dintre ele.

1.4.13 Sinteză comparativă

Tabelul 1.1 rezumă metodele prezentate, grupându-le după ideea principală și relevanța pentru extragerea relațiilor între entități. Se observă o tendință clară în literatură: de la pipeline-uri liniare fixe (Naive RAG) spre sisteme care evaluează și corectează adaptiv calitatea recuperării (CRAG, Self-RAG) sau care structurează explicit cunoștințele sub formă de graf (GraphRAG, HippoRAG). Pentru extragerea relațiilor, metodele cele mai promițătoare sunt cele care fie recuperează precis termenii rari (recuperare hibridă, ColBERT), fie modelează explicit legăturile dintre entități (abordările pe grafuri).

1.5 Metrici de evaluare

Evaluarea unui sistem RAG este mai complexă decât evaluarea unui model generativ izolat, deoarece performanța globală depinde atât de componenta de recuperare, cât și de cea de generare [2]. O recuperare slabă limitează calitatea răspunsului indiferent de capacitățile modelului generativ.

Din acest motiv, evaluarea se face de obicei pe două niveluri. Evaluarea pe componente măsoară separat calitatea recuperării (cât de bune sunt fragmentele aduse) și calitatea generării (cât de bun este răspunsul dat un context). Evaluarea end-to-end măsoară calitatea răspunsului final, așa cum îl primește utilizatorul. Cele două perspective sunt complementare: un scor end-to-end slab nu spune unde este problema, în timp ce metricile pe componente localizează cauza, dar nu garantează că un sistem cu ambele componente bune produce și un răspuns final bun. Demonstratorul din această lucrare folosește ambele niveluri, cu metrici de recuperare și metrici de generare raportate separat.

Metodă	Idee principală	Relevanță pentru extragerea relațiilor
Naive RAG [5]	Recuperare densă + generare	Referință de bază; ratează termeni rari
Advanced RAG [2]	Optimizări pre/post-recuperare	Rerankingul aduce fragmentele cu ambele entități
ColBERT [12]	Interacțiune târzie la nivel de token	Potrivire precisă a mențiunilor de entitate
HyDE [13]	Pseudo-document generat	Îmbunătățește recall fără date etichetate
RAPTOR [17]	Arbore de rezumate ierarhice	Agregare de dovezi multi-document
CRAG [18]	Evaluator al calității recuperării	Reduce extragerea din context irelevant
Self-RAG [10]	Auto-reflecție prin tokenuri speciale	Verifică susținerea relației de către dovezi
Adaptive-RAG [19]	Strategie de recuperare adaptivă	Recuperare iterativă pentru relații multi-hop
FLARE [20]	Recuperare activă pe parcursul generării	Reduce halucinațiile pe texte lungi
GraphRAG [21]	Graf de cunoștințe din corpus	Modelează explicit relațiile între entități
HippoRAG [22]	Graf + Personalised PageRank	Descoperă relații implicite distribuite

Tabel 1.1: Sintează comparativă a metodelor RAG prezentate și a relevanței lor pentru extragerea relațiilor între entități.

1.5.1 Metrici de recuperare

Precizia la rang k (Precision@ k) este proporția documentelor relevante printre primele k rezultate recuperate:

$$\text{Precision@}k = \frac{|\{\text{relevante}\} \cap \{\text{top-}k\}|}{k}. \quad (1.2)$$

Recuperarea la rang k (Recall@ k) reprezintă proporția tuturor documentelor relevante care au fost efectiv recuperate:

$$\text{Recall@}k = \frac{|\{\text{relevante}\} \cap \{\text{top-}k\}|}{|\{\text{relevante}\}|}. \quad (1.3)$$

Mean Reciprocal Rank (MRR) măsoară rangul primului document relevant, mediat peste $|Q|$ interogări:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rang}_i}. \quad (1.4)$$

Hits@ k este o metrică binară care indică dacă cel puțin un document relevant apare printre primele k rezultate. Spre deosebire de Recall@ k , care măsoară proporția documentelor relevante recuperate, Hits@ k răspunde doar la întrebarea dacă recuperarea a reușit sau nu. Este utilă atunci când un singur document relevant este suficient pentru a răspunde, situație frecventă în răspunsul la întrebări factuale.

Normalized Discounted Cumulative Gain (nDCG) ține cont de poziția exactă a fiecărui document relevant, aplicând o reducere logaritmică pentru pozițiile inferioare. Spre deosebire de MRR, care privește doar primul document relevant, nDCG evaluează întreaga listă de rezultate, fiind potrivită atunci când mai multe documente relevante contribuie la răspuns. Valoarea este normalizată între 0 și 1, unde 1 corespunde ordinii ideale.

Aceste metrice sunt calculate față de un set de documente relevante adnotate manual. În practică, Recall@ k este adesea mai informativă decât Precision@ k pentru sistemele RAG, deoarece un document relevant ratat nu poate fi recuperat ulterior de modelul generativ [7].

Un exemplu numeric clarifică aceste definiții. Să presupunem o interogare cu trei documente relevante în corpus, pentru care sistemul returnează cinci rezultate, dintre care relevante sunt cele de pe pozițiile 1, 2 și 4. Atunci Precision@5 = 3/5 = 0,6, deoarece trei din cele cinci rezultate sunt relevante, iar Recall@5 = 3/3 = 1,0, deoarece toate documentele relevante au fost recuperate. Primul rezultat relevant apare pe poziția 1, deci contribuția la MRR pentru această interogare este 1/1 = 1,0. Dacă, în schimb, primul rezultat relevant ar fi apărut pe poziția 3, contribuția ar fi fost 1/3 ≈ 0,33, ceea ce arată cum MRR penalizează plasarea fragmentului corect pe poziții inferioare.

1.5.2 Metrici de generare

Exact Match (EM) este o metrică binară care indică potrivirea completă, token cu token, față de răspunsul de referință. Este strictă și potrivită pentru răspunsuri scurte sau factuale.

Scorul F_1 la nivel de token echilibrează precizia P și recuperarea R față de răspunsul de referință:

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}. \quad (1.5)$$

Pentru răspunsuri mai lungi, unde potrivirea exactă este irealistă, se utilizează suprapunerea de n -grame calculată cu metricile ROUGE și BLEU [2]. BLEU măsoară precizia n -gramelor, adică ce proporție din n -gramele răspunsului generat apar în referință, fiind orientată spre

precizie și folosită inițial pentru traducerea automată. ROUGE măsoară în schimb acoperirea, adică ce proporție din n -gramele referinței apar în răspunsul generat, fiind orientată spre recall și folosită frecvent pentru rezumare. Varianta ROUGE-L măsoară lungimea celui mai lung subșir comun, captând ordinea parțială a tokenurilor fără a cere potrivire pe poziții consecutive.

Aceste metrice au o limitare comună importantă: penalizează reformulările corecte. Un răspuns corect din punct de vedere semantic, dar exprimat cu alte cuvinte decât referința, primește un scor scăzut, deși este perfect valid. Acest aspect este relevant pentru sistemele RAG cu modele generative moderne, care tind să parafrazeze răspunsul în loc să copieze textual fragmentul recuperat.

1.5.3 Metrice specifice sistemelor RAG

Metricile clasice de generare nu surprind fenomenul halucinației în contextul RAG, deoarece nu verifică dacă răspunsul este fundamentat pe contextul recuperat. Cadrul RAGAS [27] introduce metrice dedicate:

Fidelitatea (faithfulness) măsoară proporția afirmațiilor din răspuns care pot fi deduse din contextul recuperat. Răspunsul este descompus în afirmații individuale, iar fiecare afirmație este verificată față de context. Formal, fidelitatea este raportul

$$\text{Fidelitate} = \frac{\text{numărul de afirmații susținute de context}}{\text{numărul total de afirmații din răspuns}}. \quad (1.6)$$

O fidelitate de 1,0 indică un răspuns complet susținut de dovezile furnizate, în timp ce o valoare scăzută semnalează prezența halucinațiilor.

Relevanța răspunsului (answer relevancy) măsoară în ce grad răspunsul adresează întrebarea originală, penalizând răspunsurile complete factual, dar tangențiale față de întrebare. Se calculează generând mai multe întrebări posibile pornind de la răspuns și măsurând similaritatea lor medie cu întrebarea originală: un răspuns relevant produce întrebări apropiate de cea reală.

Precizia contextului și recuperarea contextului evaluează calitatea recuperării din perspectiva generării: câte dintre fragmentele recuperate sunt cu adevărat necesare și câte dintre fragmentele necesare au fost recuperate [27]. Aceste metrice sunt centrale pentru sistemele orientate spre fidelitate [10]; evaluarea cu RAGAS a demonstratorului din această teză este prevăzută ca extensie a analizei cantitative.

Aceste metrice au în comun faptul că folosesc un model de limbaj ca evaluator (eng. *LLM-as-judge*). În loc să compare răspunsul cu o referință fixă, un model de limbaj apreciază dacă afirmațiile din răspuns sunt susținute de context sau dacă răspunsul adresează întrebarea. Avantajul este flexibilitatea: metrica tolerează reformulările și surprinde nuanțe pe care potrivirea de n -grame le ratează. Dezavantajul este costul și dependența de calitatea modelului-judecător, care poate la rândul lui greși. Pentru extragerea relațiilor, evaluarea cu model-judecător poate verifica dacă relația exprimată în răspuns este efectiv susținută de fragmentul recuperat, ceea ce o face complementară metricilor clasice bazate pe potrivire exactă.

1.5.4 Dificultăți în evaluarea extragerii relațiilor

Evaluarea extragerii relațiilor se realizează cu scorurile micro și macro F_1 calculate pe tipurile de relații. Micro- F_1 ponderează fiecare instanță egal, favorizând tipurile de relații frecvente. Macro- F_1 tratează fiecare tip de relație egal, indiferent de frecvența sa.

Există două dificultăți structurale. Prima este distribuția dezechilibrată a tipurilor de relații în seturile de date existente, care face ca micro- F_1 să fie adesea înșelătoare [28]. A doua este

problema falselor negative: relații corecte neanotate în setul de referință duc la subestimarea performanței reale. Re-DocRED [28] a demonstrat că DocRED [29] conținea un număr semnificativ de astfel de false negative.

Metricile de fidelitate nu reflectă direct corectitudinea tipului de relație extras: un sistem poate fi fidel față de contextul recuperat, dar contextul însuși poate conține relații ambigue sau greșit formulate. Prin urmare, evaluarea completă a unui sistem RAG pentru extragerea relațiilor necesită atât metrici de fidelitate, cât și metrici specifice extragerii.

O dificultate suplimentară ține de nivelul de detaliu al evaluării. O relație poate fi corectă ca tip, dar greșită ca argumente, sau corectă ca argumente, dar greșită ca direcție (de exemplu, „X este autorul lui Y” față de „Y este autorul lui X”). O metrică binară care marchează relația ca fiind corectă sau greșită pierde aceste nuanțe. De aceea, evaluările riguroase raportează separat performanța pe identificarea entităților, pe clasificarea tipului de relație și pe direcția relației, ceea ce oferă o imagine mai detaliată a punctelor slabe ale sistemului.

1.6 Baze de date

Evaluarea unui sistem RAG pentru extragerea relațiilor se sprijină pe două tipuri de seturi de date. Primul tip provine din regăsirea informației și răspunsul la întrebări (QA), unde se măsoară dacă sistemul recuperează pasajul corect și produce răspunsul potrivit. Al doilea tip este specific extragerii relațiilor, unde adnotările sunt triplete de forma (subiect, relație, obiect) și se măsoară dacă sistemul identifică legăturile corecte dintre entități. Această secțiune prezintă seturile reprezentative din ambele categorii, iar Tabelul 1.2 le sintetizează. Demonstratorul din această lucrare este validat inițial pe SQuAD, urmând ca evaluarea pe extragerea relațiilor să folosească seturi precum T-REx.

1.6.1 Seturi de date pentru sisteme RAG

SQuAD [30] (Stanford Question Answering Dataset) leagă întrebări de pasaje din Wikipedia, răspunsul reprezentând un fragment de text contiguu extras din pasaj. Setul conține 100.000 de întrebări construite de adnotatori umani, ceea ce îl face un standard pentru evaluarea recuperării și extracției.

Natural Questions [31] conține întrebări reale adresate motorului de căutare Google, perechi cu pagini Wikipedia. Spre deosebire de SQuAD, întrebările nu sunt construite cu pasaje în față, ceea ce le face mai reprezentative pentru scenariile reale de utilizare. Setul de date include atât răspunsuri scurte, cât și lungi.

TriviaQA [32] este un set de perechi întrebare-răspuns colectate din resurse trivia, însoțite de documente justificative găsite independent prin căutare web. Dificultatea suplimentară față de SQuAD este că documentele justificative nu sunt garantat să conțină răspunsul.

HotpotQA [33] este un benchmark care necesită raționament pe mai multe documente simultan. Fiecare întrebare este adnotată cu faptele justificative din mai multe articole Wikipedia, forțând sistemele să realizeze recuperare și raționament multi-hop. Este relevant pentru extragerea relațiilor care implică mai multe entități intermediare.

CRAG [34] (Comprehensive RAG Benchmark) conține 4.409 perechi QA generate cu API-uri de căutare web și de grafuri de cunoștințe. Acoperă cinci domenii și opt categorii de întrebări cu niveluri variabile de dinamism temporal, ceea ce îl face util pentru evaluarea robusteții față de informații care se schimbă în timp.

1.6.2 Seturi de date pentru extragerea relațiilor

T-REx [35] este un set de aliniament la scară largă între rezumatele Wikipedia și tripletele Wikidata: 11 milioane de triplete aliniate cu 3,09 milioane de rezumate. Structura sa face T-REx potrivit pentru evaluarea sistemelor care combină recuperarea din text liber cu validarea față de grafuri de cunoștințe. Reprezintă setul de date vizat pentru evaluarea pe extragerea relațiilor din această lucrare.

WebNLG [36] furnizează perechi date-text pornind de la triplete RDF din DBpedia. Setul de date are aplicații atât în extragerea relațiilor, cât și în verbalizarea datelor structurate, deoarece fiecare triplet este însoțit de fraze naturale care îl exprimă.

SemEval-2010 Task 8 [37] este un benchmark de clasificare a relațiilor semantice între perechi nominale la nivel de propoziție, cu nouă tipuri de relații direcționale plus o clasă negativă. Este utilizat frecvent ca referință pentru compararea metodelor de extragere a relațiilor la nivel de propoziție.

TACRED [38] este un set de date de mari dimensiuni construit din articole de știri și rapoarte ale agenției Associated Press, cu 41 de tipuri de relații și o clasă negativă predominantă. Dezechilibrul accentuat între clase face evaluarea macro- F_1 mai informativă decât micro- F_1 .

DocRED [29] extinde extragerea relațiilor la nivel de document, necesitând raționament inter-propoziții. Un număr semnificativ de relații implică entități menționate în propoziții diferite, ceea ce face DocRED mai dificil decât seturile sentence-level.

Re-DocRED [28] corectează problema falselor negative din DocRED prin reannotare manuală sistematică. Studiul original a arătat că o parte considerabilă din relațiile corecte din DocRED nu fuseseră adnotate, ceea ce subestima performanța modelelor evaluate pe setul original.

Set de date	Sarcină	Nivel	Caracteristici principale
SQuAD	QA extractiv	Propoziție	Fragment de text ca răspuns
Natural Questions	QA	Document	Întrebări reale, răspuns scurt/lung
HotpotQA	QA multi-hop	Multi-document	Fapte justificative adnotate
CRAG	QA factual	Web / GC	4.409 perechi, dinamism temporal
T-REx	Extragere relații	Document	Aliniament text-triplete Wikidata
WebNLG	Extragere/verbalizare	Triplete RDF	Perechi date-text
SemEval-2010 T8	Extragere relații	Propoziție	Relații nominale, 9 tipuri
TACRED	Extragere relații	Propoziție	41 tipuri de relații
DocRED	Extragere relații	Document	Raționament inter-propoziții
Re-DocRED	Extragere relații	Document	Corectare false negative

Tabel 1.2: Comparatie a seturilor de date pentru evaluarea RAG și extragerea relațiilor [28–31, 33–38].

Alegerea setului de date pentru evaluare depinde de ceea ce se dorește măsurat. Pentru a valida recuperarea și răspunsul la întrebări, seturile de tip QA precum SQuAD sunt potrivite, deoarece răspunsul este un fragment cu sursă cunoscută, ceea ce permite o verificare automată și obiectivă. Pentru a măsura direct extragerea relațiilor, sunt necesare seturi cu triplete adnotate, precum T-REx sau DocRED, unde se poate verifica dacă sistemul identifică legătura corectă dintre două entități.

În această lucrare, validarea inițială folosește SQuAD, tocmai pentru că permite o măsurare precisă și reproductibilă a recuperării, fără a depinde de un model-judecător. Evaluarea pe extragerea relațiilor, pe seturi de tip T-REx, este pasul următor: arhitectura validată pe regăsire și răspuns la întrebări este transferată pe sarcina-țintă, unde adnotările sub formă de triplete permit o evaluare directă a relațiilor extrase.

1.7 Limitări și provocări

Calitatea recuperării. Documentele irelevante sau parțial relevante induc în eroare modelul generativ, ducând la halucinații chiar și atunci când există o sursă externă de informație [2, 18]. Recuperarea fixă, neadaptivă, nu poate evalua sau corecta automat rezultatele slabe [10]. Fragmentele care conțin una dintre entitățile căutate, dar nu și relația relevantă, sunt o sursă frecventă de erori în extragerea relațiilor.

Integrarea contextelor lungi. Creșterea numărului de pasaje recuperate nu îmbunătățește neapărat performanța. Pasajele irelevante din contexte lungi degradează calitatea textului generat [11]. Fenomenul, denumit uneori *lost in the middle*, arată că modelele generative tind să ignore informațiile aflate la mijlocul unui context lung, chiar dacă acestea sunt relevante.

Limitări ale datelor și metricilor. Seturile de date pentru extragerea relațiilor au distribuții dezechilibrate și conțin false negative [28]. Metricile de fidelitate nu reflectă direct corectitudinea tipului de relație extras [27]. Construirea de seturi de date de calitate la nivel de document este costisitoare și lentă.

Dificultăți în extragerea relațiilor. Identificarea unei relații necesită adesea agregarea dovezilor din mai multe propoziții sau documente, ceea ce presupune un raționament inter-propoziție dificil de realizat într-un singur pas de recuperare [21, 29]. Relațiile implicite, care nu apar formulat explicit în text, sunt o provocare suplimentară.

Costul computațional. Indexarea densă, recuperarea hibridă, rerankingul cu cross-encoder și generarea cu LLM-uri sunt toate operații care consumă resurse semnificative [7, 16]. Abordările adaptive și active implică recuperare iterativă, cu mai multe apeluri la model pe interogare [19, 20]. Metodele bazate pe grafuri adaugă costul construirii și actualizării grafului de cunoștințe [17, 22]. În absența unor resurse computaționale ample, aceste cerințe limitează scala la care sistemele pot fi evaluate sau folosite în producție.

Dependența de evaluarea automată. Multe metrici specifice RAG, precum cele din cadrul RAGAS [27], folosesc un model de limbaj ca judecător pentru a aprecia fidelitatea sau relevanța răspunsului. Această abordare introduce o dependență circulară: calitatea evaluării depinde de calitatea unui alt model de limbaj, care la rândul lui poate halucina sau aprecia greșit. În plus, evaluarea cu un model-judecător este lentă și costisitoare, deoarece presupune câte un apel suplimentar pentru fiecare pereche întrebare-răspuns evaluată, ceea ce face dificilă rularea pe seturi mari de date.

Securitate și surse nesigure. Atunci când baza de cunoștințe conține informații greșite, învechite sau intenționat manipulate, sistemul RAG le poate prelua și prezenta ca fapte, cu aparența de autoritate dată de fundamentarea pe o sursă. Robustețea unui sistem RAG nu depinde doar de arhitectură, ci și de calitatea și de fiabilitatea sursei de cunoștințe, un aspect adesea trecut cu vederea în evaluările de laborator.

1.8 Concluzii

Capitolul de față a prezentat contextul teoretic al sistemelor RAG aplicate extragerii relațiilor între entități. Au fost discutate limitările cunoștințelor parametrice ale modelelor lingvistice mari [2, 5] și motivul pentru care augmentarea cu recuperare externă este o soluție practică, urmată de o taxonomie specifică acestei teze, construită pe criteriile tipului de mecanism de recuperare, al modului de integrare recuperare-generare și al sursei de cunoștințe.

Analiza literaturii arată că robustețea sistemelor RAG depinde în primul rând de calitatea recuperării și de capacitatea de a evalua adaptiv dovezile recuperate [10, 18]. Extragerea relațiilor beneficiază cel mai mult de structurile de grafuri de cunoștințe și de abordările cu raționament la nivel de document [21, 22, 29]. Provocările deschise rămân integrarea contextelor

lungi [11], limitările metricilor și datelor [27, 28] și costul computațional [16].

Lucrarea se plasează la intersecția optimizării recuperării cu dezvoltarea de mecanisme generative care integrează explicit dovezile recuperate. Demonstratorul propus compară Naive RAG cu Advanced RAG, pornind de la observația că optimizările pre- și post-recuperare conduc la îmbunătățiri măsurabile ale fidelității și relevanței răspunsurilor [2, 27], aspecte cu impact direct asupra corectitudinii relațiilor extrase.

Direcția adoptată este motivată de un compromis practic. Arhitecturile bazate pe grafuri sunt cele mai potrivite structural pentru extragerea relațiilor, dar presupun un cost ridicat de construire a grafului și o complexitate de implementare semnificativă. Advanced RAG, prin recuperarea hibridă și rerankingul aplicat pe text nestructurat, oferă o parte importantă din beneficii la un cost mult mai mic, ceea ce îl face un punct de plecare rezonabil pentru un demonstrator. Capitolul următor descrie implementarea acestei arhitecturi și o compară cantitativ și calitativ cu varianta de bază.

Capitolul 2

Rezultate experimentale

Partea aplicativă a lucrării constă într-un demonstrator care pune față în față două arhitecturi RAG: una de bază (Naive RAG) și una cu optimizări de recuperare (Advanced RAG). Ideea de pornire a fost simplă. Dacă vrem să măsurăm cât ajută optimizările de recuperare descrise în Capitolul 1, cel mai curat experiment este să ținem totul constant între cele două sisteme, cu o singură excepție: complexitatea etapei de recuperare. Ambele folosesc același model generativ, același set de date și aceeași bază de cunoștințe. Diferă doar prin felul în care aleg fragmentele pe care le trimit modelului.

Sistemul rulează integral local, pe hardware propriu, fără apeluri către servicii externe. Această alegere a fost importantă din două motive. Primul ține de reproducibilitate: un model rulat local produce aceleași rezultate de fiecare dată, fără a depinde de versiunea unui API care se poate schimba peste noapte. Al doilea ține de control: avem acces complet la promptul de sistem, la parametrii de recuperare și la fragmentele intermediare, ceea ce face posibilă analiza pas cu pas a comportamentului.

Acest capitol descrie mai întâi proiectarea sistemului, adică cerințele, tehnologiile, arhitectura și algoritmiile celor două pipeline-uri, împreună cu interfața și fluxul utilizatorului. Urmează analiza cantitativă, în care cele două sisteme sunt comparate pe metrici de recuperare și de generare, și analiza calitativă, care interpretează rezultatele și discută compromisurile observate.

2.1 Proiectarea sistemului

2.1.1 Arhitectura generală

Sistemul are două faze. Faza offline, executată o singură dată, construiește baza de cunoștințe din setul de date. Faza online, executată la fiecare interogare, recuperează fragmentele relevante și generează răspunsul. Codul este împărțit în module care corespund acestor faze: `ingest.py` pentru indexare, pachetele `naive` și `advanced` pentru cele două pipeline-uri, `benchmark.py` pentru evaluare și `main.py` pentru interfață.

Figura 2.1 prezintă faza offline. Setul de date este încărcat, paragrafele sunt deduplicate, apoi codificate și depuse în două indexuri: unul dens în ChromaDB și unul rar, BM25. Ambele pipeline-uri citesc din aceleași indexuri, ceea ce garantează că diferențele de performanță provin din strategia de recuperare, nu din date diferite. Cele două pipeline-uri și fluxul lor de procesare sunt descrise în detaliu mai jos, după prezentarea cerințelor și a tehnologiilor.

2.1.2 Cerințe

Înainte de implementare au fost stabilite câteva cerințe care au ghidat deciziile ulterioare.

Pe partea funcțională, sistemul trebuie să implementeze două pipeline-uri RAG complete, de la interogare la răspuns, peste aceeași bază de cunoștințe. Trebuie să existe o interfață care permite interogarea simultană a ambelor sisteme și afișarea răspunsurilor una lângă alta, astfel încât diferențele să fie vizibile direct. În plus, este nevoie de un modul de evaluare automată care rulează cele două sisteme pe un set de întrebări și calculează metrici comparative.

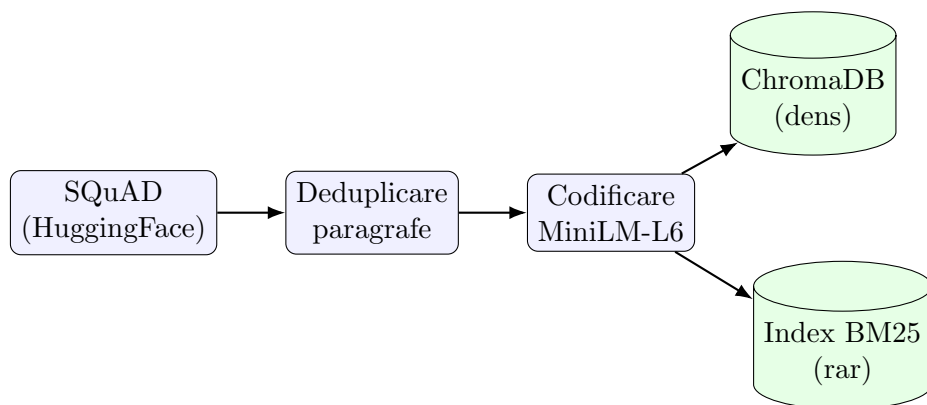


Figura 2.1: Faza offline de indexare. Paragrafele unice din SQuAD sunt codificate o singură dată și depuse în indexul dens (ChromaDB) și în indexul rar (BM25), folosite în comun de ambele pipeline-uri.

Pe partea nefuncțională, comparația trebuie să fie corectă, ceea ce înseamnă că singura variabilă între cele două sisteme este arhitectura de recuperare. Rezultatele trebuie să fie reproducibile, motiv pentru care evaluarea rulează local, cu seed-uri fixate. Sistemul trebuie să ruleze pe o singură placă grafică de consum, fără infrastructură distribuită. În fine, codul trebuie organizat modular, cu pipeline-urile separate de interfață și de modulul de evaluare, pentru ca fiecare componentă să poată fi modificată independent.

2.1.3 Tehnologii utilizate

Tabelul 2.1 rezumă tehnologiile folosite și rolul fiecăreia. Alegerile au urmărit instrumente mature, larg folosite în sistemele RAG, care pot rula local.

Tabel 2.1: Tehnologiile utilizate în demonstrator

Componentă	Tehnologie	Rol
Model generativ	Ollama + Qwen3-8B	Generarea răspunsului
Expansiune interogări	Qwen3-1.7B	Reformularea interogării
Model de embedding	all-MiniLM-L6-v2	Codificarea semantică a textului
Reranking	ms-marco-MiniLM-L-6-v2	Re-scorarea fragmentelor
Bază vectorială	ChromaDB	Indexare și căutare densă
Indexare rară	bm25s	Căutare BM25 pe cuvinte cheie
Interfață	Streamlit	Interacțiune și vizualizare
Evaluare	RAGAS, SciPy	Metrici și teste statistice
Set de date	SQuAD (HuggingFace)	Corpus și întrebări de test

Modelul generativ este Qwen3-8B [2], rulat prin Ollama. Qwen3 are un mod de raționament (“thinking mode”) care poate fi dezactivat cu directiva `/no_think` în prompt. Această opțiune este folosită la expansiunea interogărilor, unde un raționament extins ar încetini procesul fără să aducă un beneficiu real.

Modelul de embedding este `all-MiniLM-L6-v2` [7], un model compact de aproximativ 22 de milioane de parametri care produce vectori de dimensiune 384. Raportul bun între calitate și viteză îl face o alegere des întâlnită în sistemele RAG fără constrângeri hardware mari. Pentru reranking se folosește un cross-encoder din aceeași familie, `ms-marco-MiniLM-L-6-v2`, antrenat pentru a estima relevanța unei perechi (interogare, pasaj).

2.1.4 Configurația hardware

Întreg sistemul a fost dezvoltat și evaluat pe o singură stație de lucru, fără infrastructură distribuită. Configurația cuprinde o placă grafică NVIDIA RTX A5000 cu 24 GB de memorie video, un procesor Intel Xeon Silver 4309Y la 2,80 GHz cu 8 nuclee și 16 fire de execuție, 64 GB de memorie RAM și un disc M.2 de 1 TB. Placa grafică rulează atât modelele de embedding și cross-encoder, cât și cele două modele Qwen3 prin Ollama. Cei 24 GB de memorie video sunt suficienți pentru a ține simultan modelul generativ de 8B parametri și modelele auxiliare, fără a le descărca și reîncărca între interogări. Toate valorile de latență raportate în acest capitol au fost măsurate pe această configurație și depind de ea.

2.1.5 Indexarea datelor

Indexarea este realizată de scriptul `ingest.py` și rulează o singură dată, la instalare. Setul de date SQuAD este încărcat din depozitul HuggingFace, iar paragrafele se repetă, pentru că fiecare apare în mai multe rânduri, câte unul per întrebare. Primul pas elimină aceste duplicate, păstrând fiecare paragraf o singură dată împreună cu titlul articolului din care provine.

Unitatea de bază a corpusului este paragraful SQuAD, indexat ca atare. Nu se aplică o fragmentare suplimentară, deoarece paragrafele SQuAD au deja o lungime potrivită pentru recuperare, suficient de scurte pentru a fi specifice și suficient de lungi pentru a păstra contextul răspunsului. Paragrafele unice sunt codificate în loturi de 256 cu modelul MiniLM și depuse în ChromaDB, configurat cu spațiu cosinus pentru similaritate. În paralel, textul brut al acelorași paragrafe este tokenizat și indexat cu BM25. Indexul BM25 se construiește la prima utilizare și se salvează pe disc, astfel încât pornirile ulterioare să nu îl reconstruiască.

2.1.6 Pipeline-ul Naive RAG

Naive RAG urmează cel mai simplu traseu posibil, cel din formularea originală a RAG [5]: codifică interogarea, caută cele mai apropiate fragmente prin similaritate cosinus și le trimite modelului generativ. Nu există reformulare, recuperare hibridă sau reranking. Pașii, ilustrați în Figura 2.2, sunt următorii.

1. Interogarea este codificată cu `all-MiniLM-L6-v2` într-un vector de dimensiune 384.
2. ChromaDB returnează primele $K = 5$ fragmente după similaritate cosinus.
3. Fragmentele sunt concatenate și incluse într-un prompt împreună cu interogarea.
4. Qwen3-8B generează răspunsul, instruit prin promptul de sistem să folosească exclusiv contextul furnizat.

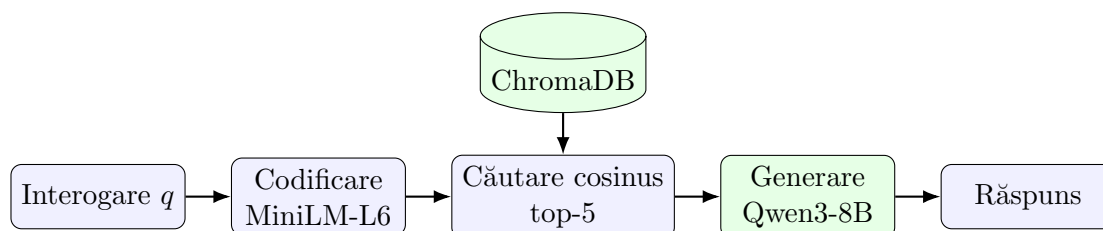


Figura 2.2: Fluxul de procesare Naive RAG. Interogarea este codificată, cele mai apropiate cinci fragmente sunt căutate prin similaritate cosinus în ChromaDB și trimise direct modelului generativ, fără reformulare, recuperare hibridă sau reranking.

Promptul de sistem cere modelului să răspundă doar pe baza contextului și să refuze explicit dacă informația lipsește. Acest baseline are limitările cunoscute din literatură [2]: ratează termenii rari pe care similaritatea semantică îi diluează și nu filtrează fragmentele irelevante recuperate din greșeală.

2.1.7 Pipeline-ul Advanced RAG

Advanced RAG adaugă patru etape peste varianta de bază: expansiunea interogării, recuperarea hibridă, fuziunea rezultatelor și rerankingul. Figura 2.3 prezintă fluxul complet, iar Tabelul 2.2 listează parametrii folosiți.

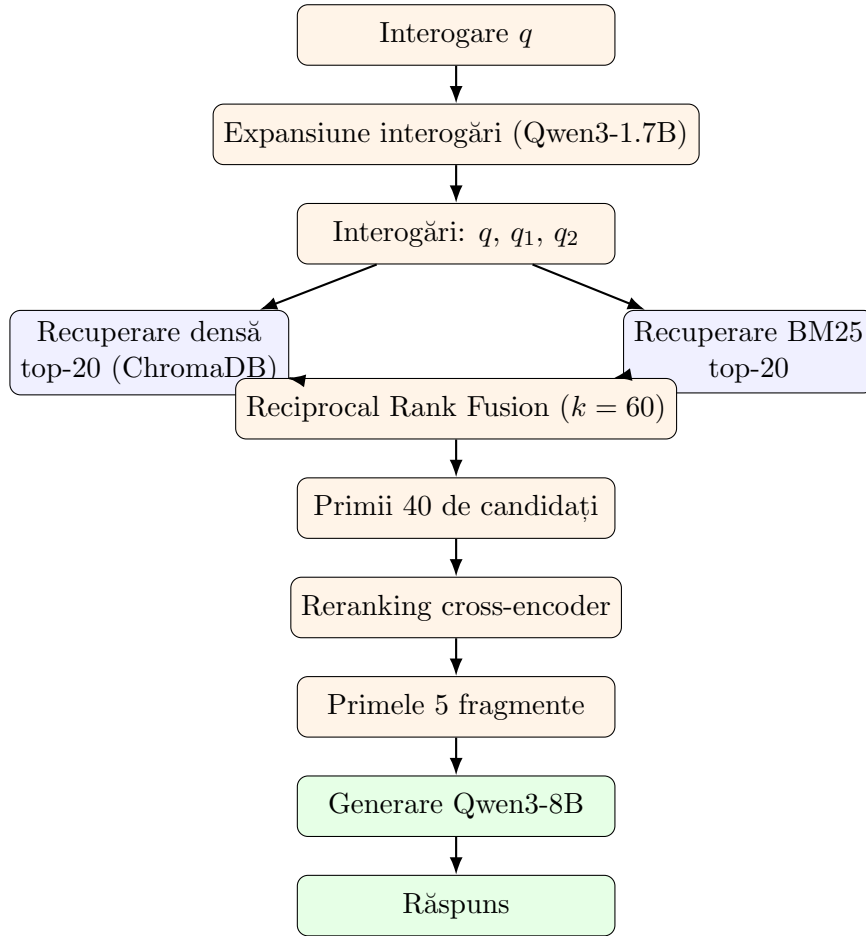


Figura 2.3: Fluxul de procesare Advanced RAG. Interogarea este reformulată, fiecare variantă este căutată dens și prin BM25, rezultatele sunt fuzionate prin Reciprocal Rank Fusion, re-scorate cu un cross-encoder, iar primele cinci fragmente sunt trimise modelului generativ.

Prima etapă este expansiunea interogării. Un model mic, Qwen3-1.7B, generează două reformulări ale interogării originale. Scopul este reducerea dependenței de formularea exactă a întrebării: dacă utilizatorul folosește alți termeni decât cei din document, o reformulare bună poate recupera totuși fragmentul corect. Modelul mic a fost ales deliberat, pentru că reformularea nu cere raționament complex, iar acest apel se află pe calea critică a fiecărei interogări.

A doua etapă este recuperarea hibridă. Pentru fiecare dintre cele trei interogări (originala și cele două reformulări) se execută în paralel o căutare densă în ChromaDB și o căutare BM25, fiecare returnând câte 20 de candidați. Recuperarea densă prinde asemănările de sens, iar BM25 [8] prinde potrivirile lexicale exacte, utile pentru termeni rari, numere și denumiri

Tabel 2.2: Parametrii pipeline-ului Advanced RAG

Parametru	Valoare	Semnificație
Reformulări ale interogării	2	Variante generate pe lângă interogarea originală
Candidați denși per variantă	20	Fragmente returnate de ChromaDB
Candidați BM25 per variantă	20	Fragmente returnate de BM25
Constanta RRF	60	Parametrul k din fuziune
Dimensiunea pool-ului	40	Candidați trimiși la reranking
Fragmente finale	5	Fragmente trimise modelului generativ

proprii. Cele șase căutări rulează concurrent, pe fire de execuție separate, pentru a limita timpul total.

A treia etapă fuzionează cele șase liste de rezultate prin Reciprocal Rank Fusion [9]. Fiecare fragment primește un scor care însumează contribuțiile din toate listele în care apare, după formula

$$\text{RRF}(d) = \sum_l \frac{1}{k + r_l(d)}, \quad (2.1)$$

unde $r_l(d)$ este rangul fragmentului d în lista l , iar $k = 60$. Constanta k atenuează influența pozițiilor de top, astfel încât un fragment aflat constant pe locuri bune în mai multe liste să fie favorizat față de unul aflat pe primul loc într-o singură listă. După fuziune se păstrează primii 40 de candidați.

A patra etapă este rerankingul. Cross-encoderul `ms-marco-MiniLM-L-6-v2` primește perechea (interogare, fragment) și produce un scor de relevanță. Spre deosebire de modelul de embedding, care codifică separat interogarea și fragmentul, cross-encoderul le procesează împreună, ceea ce dă o estimare mai precisă a relevanței, dar cu un cost de calcul mai mare. Din acest motiv el se aplică doar celor 40 de candidați rămași, nu întregului corpus. Primele cinci fragmente după rerankingul cross-encoder ajung la modelul generativ, care produce răspunsul cu același prompt de grounding ca în varianta Naive.

2.1.8 Interfața și fluxul utilizatorului

Interfața este construită cu Streamlit și are forma unei conversații. Utilizatorul scrie o întrebare, iar răspunsurile celor două sisteme apar una lângă alta, pe două coloane, fiecare cu timpul de generare afișat deasupra. Această așezare face comparația imediată: pe aceeași întrebare se vede direct cum diferă răspunsurile și cât durează fiecare.

La pornire, interfața încarcă o singură dată modelele și indexul BM25 și le păstrează în memorie printr-un mecanism de cache. Fără acest pas, fiecare interogare ar reîncărca modelele de pe disc, ceea ce ar face interacțiunea inutilizabil de lentă. Când utilizatorul trimite o întrebare, sistemul recuperează fragmentele pentru ambele pipeline-uri, apoi generează cele două răspunsuri în paralel, pe fire de execuție separate, ca să nu aștepte un sistem după celălalt. Răspunsurile sunt apoi adăugate în istoricul conversației, păstrat pe durata sesiunii.

Legătura dintre interfață și arhitectura RAG este directă. Coloana din stânga apelează pipeline-ul Naive, cea din dreapta pipeline-ul Advanced, iar ambele citesc din aceleași indexuri construite în faza offline. Interfața nu conține logică de recuperare proprie; ea doar orchestrează apelurile către cele două module și afișează rezultatele. Figurile 2.4 și 2.5 prezintă interfața în uz.

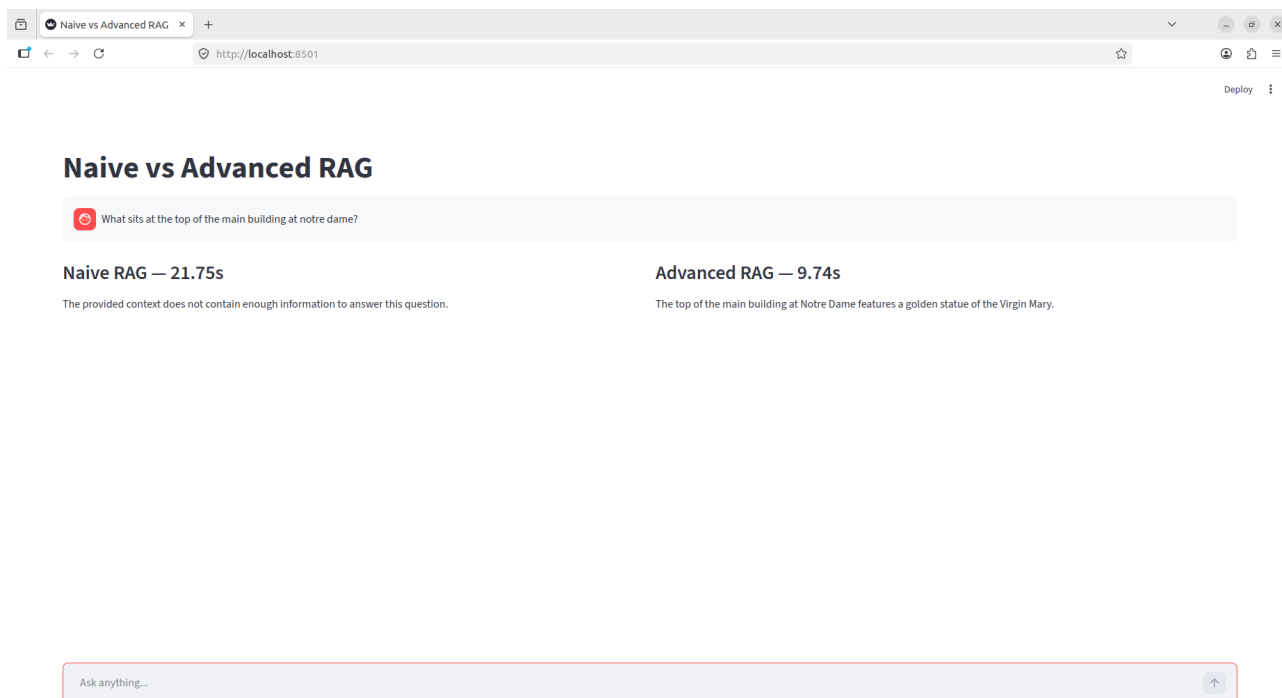


Figura 2.4: Interfața comparativă a demonstratorului. Cele două coloane corespund pipeline-urilor Naive și Advanced, care răspund simultan la aceeași întrebare, cu timpii de generare afișați deasupra fiecărui răspuns. În acest exemplu, Naive RAG nu găsește răspunsul, în timp ce Advanced RAG recuperează fragmentul corect.

2.1.9 Modulul de evaluare

Pe lângă cele două pipeline-uri și interfață, demonstratorul include un modul de evaluare automată, implementat în scriptul `benchmark.py`. Rolul lui este să ruleze ambele sisteme pe același eșantion de întrebări și să producă metricile, testele statistice și figurile raportate în acest capitol, fără intervenție manuală.

Modulul funcționează în bucle imbricate: pentru fiecare seed se extrage un eșantion de întrebări din SQuAD, iar pentru fiecare întrebare se rulează ambele pipeline-uri și se înregistrează fragmentele recuperate, timpul de procesare și, opțional, răspunsul generat. Evaluarea răspunsurilor este opțională tocmai pentru că generarea este pasul cel mai lent: metricile de recuperare se pot calcula rapid și separat, fără apeluri la modelul generativ.

O decizie de implementare care s-a dovedit utilă este salvarea de checkpoint: după fiecare seed, rezultatele parțiale se scriu pe disc printr-o înlocuire atomică de fișier, astfel încât o întrerupere în timpul rulării (care poate dura ore cu generarea activată) să nu piardă seed-urile deja terminate. O altă priviște tratarea refuzurilor la evaluarea cu RAGAS: răspunsurile în care sistemul declară explicit că informația lipsește din context sunt excluse de la metrica de relevanță, deoarece un refuz corect ar fi altfel penalizat ca un răspuns nerelevant, ceea ce ar distorsiona comparația în defavoarea sistemului mai prudent.

La final, modulul agregă rezultatele pe toate seed-urile, calculează media și deviația standard pentru fiecare metrică, rulează testele de semnificație pe perechile de rezultate și generează automat figurile comparative în format PDF și PNG, gata de inclus în lucrare.

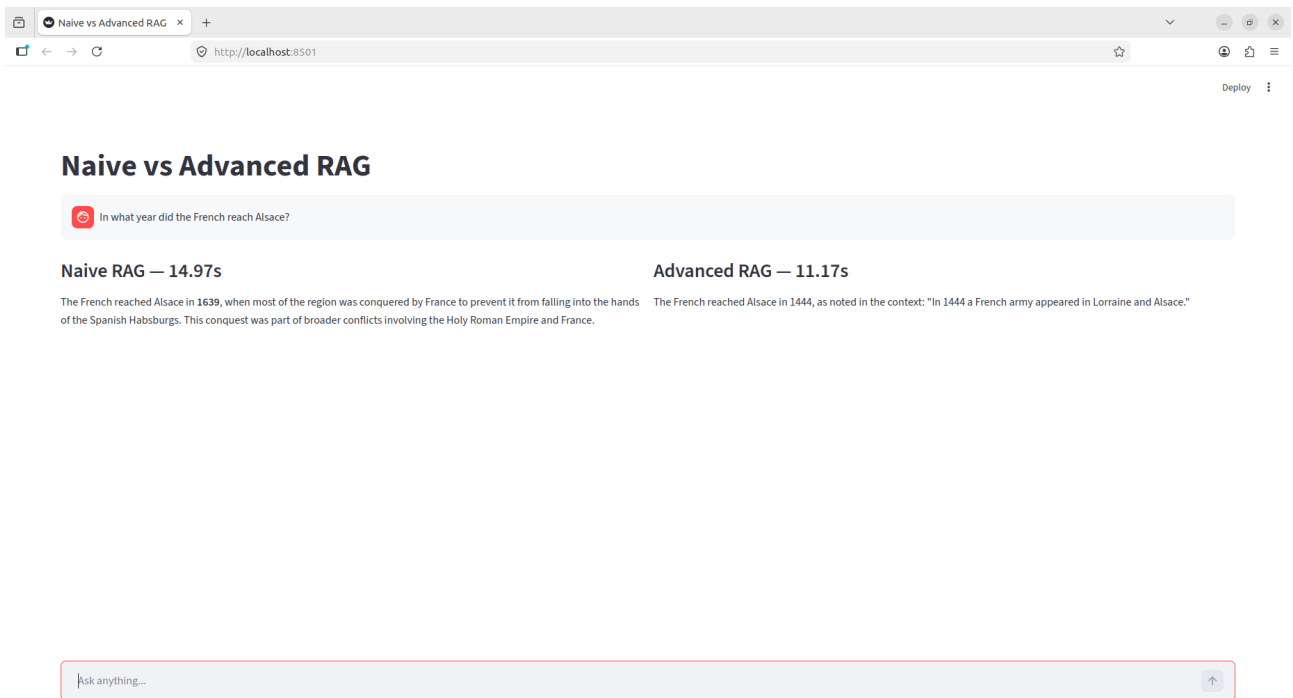


Figura 2.5: Exemplu de interogare cu termeni numerici. Naive RAG produce un răspuns numeric halucinat, deoarece similaritatea cosinus ratează valoarea exactă, în timp ce Advanced RAG găsește valoarea corectă prin recuperarea BM25.

2.1.10 Procesul de dezvoltare

Implementarea a pornit cu varianta minimă funcțională: indexarea datelor SQuAD în ChromaDB și un pipeline Naive conectat la interfața Streamlit. Abia după ce baseline-ul funcționa de la cap la coadă au fost adăugate, una câte una, componentele Advanced RAG, ca să se poată vedea la fiecare pas ce aduce fiecare optimizare.

Prima extensie a fost recuperarea sparse cu BM25 [8]. Inițial s-a folosit biblioteca `rank_bm25`, dar aceasta a fost înlocuită cu `bm25s`, mult mai rapidă prin tokenizare în loturi și indexare optimizată. Pentru că reconstruirea indexului la fiecare pornire era costisitoare, indexul se calculează o singură dată și se salvează pe disc.

Configurarea cross-encoderului a cerut câteva încercări. Au fost testate mai multe modele de reranking, inclusiv o variantă mai mare, abandonată în favoarea lui `ms-marco-MiniLM-L-6-v2`, care oferă un compromis bun între precizie și viteză. Pool-ul de candidați trimiși la reranking a crescut de la 20 la 40 pe parcurs, după ce s-a observat că un pool mai mic tăia uneori fragmentele relevante înainte ca cross-encoderul să le poată evalua.

Expansiunea interogărilor a trecut și ea prin ajustări. Modelul de reformulare a fost schimbat pe Qwen3-1.7B, iar recuperările pentru variantele de interogare au fost paralelizate, pentru că rularea lor secvențială dubla timpul de răspuns. Toate aceste decizii sunt vizibile în istoricul de versiuni al proiectului, unde fiecare modificare corespunde unui pas în rafinarea pipeline-ului.

În final, modulul de evaluare a fost construit cu suport pentru mai multe seed-uri și checkpoint după fiecare rulare, astfel încât o întrerupere să nu piardă rezultatele deja calculate. Testele de semnificație statistică (Wilcoxon și McNemar) au fost adăugate ulterior, pentru a verifica dacă diferențele observate sunt reale sau doar artefacte ale eșantionării aleatoare.

2.2 Analiză cantitativă

2.2.1 Configurarea experimentului

Evaluarea a rulat pe câte 150 de întrebări selectate aleator din split-ul de antrenare SQuAD [30], pe 4 seed-uri distincte (42, 1042, 2042, 3042), totalizând 600 de evaluări per sistem. Rularea pe mai multe seed-uri permite estimarea variabilității și raportarea mediei împreună cu deviația standard, în loc de o singură cifră care ar putea depinde de eșantionul ales.

Metricile calculate sunt:

- **Context Recall@5**: fracția de întrebări pentru care cel puțin unul din cei $K=5$ candidați recuperați conține răspunsul corect și provine din articolul corect. Condiția privind sursa este necesară pentru a preveni potrivirile false pe fragmente scurte și frecvente (de exemplu, “mai” sau “SUA”) care pot apărea în contexte complet diferite [30].
- **Mean Reciprocal Rank (MRR)**: inversul rangului primului fragment relevant, mediat pe toate întrebările [7]. MRR penalizează sistemele care plasează fragmentul corect pe poziții inferioare în lista de rezultate.
- **Answer F1**: scorul F_1 la nivel de token între răspunsul generat și răspunsurile de referință SQuAD, calculat ca maxim peste toate variantele de referință disponibile.
- **Exact Match**: proporția de răspunsuri generate care conțin textul de referință ca subsir exact, după normalizarea majusculor și a semnelor de punctuație.
- **Latența medie de recuperare**: timpul de procesare a interogării în milisecunde. Pentru Advanced RAG, aceasta include apelul LLM pentru expansiunea interogării, recuperarea hibridă paralelă și re-scorarea cu cross-encoder.

Pentru testarea semnificației statistice s-au aplicat două teste pereche pe cele 600 de înregistrări combinate din toate seed-urile. Testul Wilcoxon signed-rank a fost aplicat pe metricile continue (MRR și Answer F1), iar testul McNemar pe metricile binare (Context Recall și Exact Match) [2].

2.2.2 Asigurarea unei comparații corecte

Validitatea concluziilor depinde de corectitudinea comparației, adică de garanția că singura diferență dintre cele două sisteme este arhitectura de recuperare. Mai multe decizii de proiectare servesc acest scop.

În primul rând, ambele pipeline-uri folosesc același model generativ (Qwen3-8B), același model de embedding (all-MiniLM-L6-v2) și aceeași bază de cunoștințe, indexată o singură dată. Diferă doar etapa de recuperare. În al doilea rând, ambele sisteme primesc exact același prompt de sistem, cu aceeași constrângere de grounding, astfel încât diferențele de comportament la generare să nu provină din instrucțiuni diferite. În al treilea rând, ambele sisteme sunt evaluate pe exact aceleași întrebări, în aceeași ordine, pentru fiecare seed, ceea ce permite aplicarea testelor statistice pereche.

În fine, înainte de fiecare rulare de evaluare, modelele și indexul BM25 sunt încărcate printr-o etapă de încălzire, astfel încât timpii de latență măsurați să nu includă costul unic de inițializare. Aceste măsuri reduc riscul ca diferențele observate să fie atribuite unor factori colaterali, nu arhitecturii de recuperare.

2.2.3 Rezultate

Rezultatele agregate pe cele 4 seed-uri sunt prezentate în Tabelul 2.3. Advanced RAG depășește Naive RAG pe toate metricile de calitate, cu o creștere de 12,0 puncte procentuale la Context Recall (de la 83,5% la 95,5%), 0,185 la MRR (de la 0,719 la 0,904) și 10,3 puncte procentuale la Exact Match (de la 64,2% la 74,5%). Answer F1 crește mai modest, cu 7,3% relativ (de la 0,191 la 0,205).

Tabel 2.3: Rezultate comparative Naive RAG vs. Advanced RAG pe SQuAD ($N = 600$ per sistem, medie $\pm$ deviație standard pe 4 seed-uri)

Metrică	Naive RAG	Advanced RAG
Context Recall@5	0,835 $\pm$ 0,006	0,955 $\pm$ 0,015
MRR	0,719 $\pm$ 0,013	0,904 $\pm$ 0,023
Exact Match	0,642 $\pm$ 0,027	0,745 $\pm$ 0,045
Answer F1	0,191 $\pm$ 0,008	0,205 $\pm$ 0,015
Latență medie (ms)	15,6 $\pm$ 0,1	769,3 $\pm$ 20,8

Figurile 2.6 și 2.7 ilustrează grafic diferențele pentru metricile de recuperare și de calitate a răspunsurilor. Figura 2.8 prezintă comparativ latența medie de procesare, iar Figura 2.9 oferă o vedere de ansamblu a tuturor metricilor de calitate.

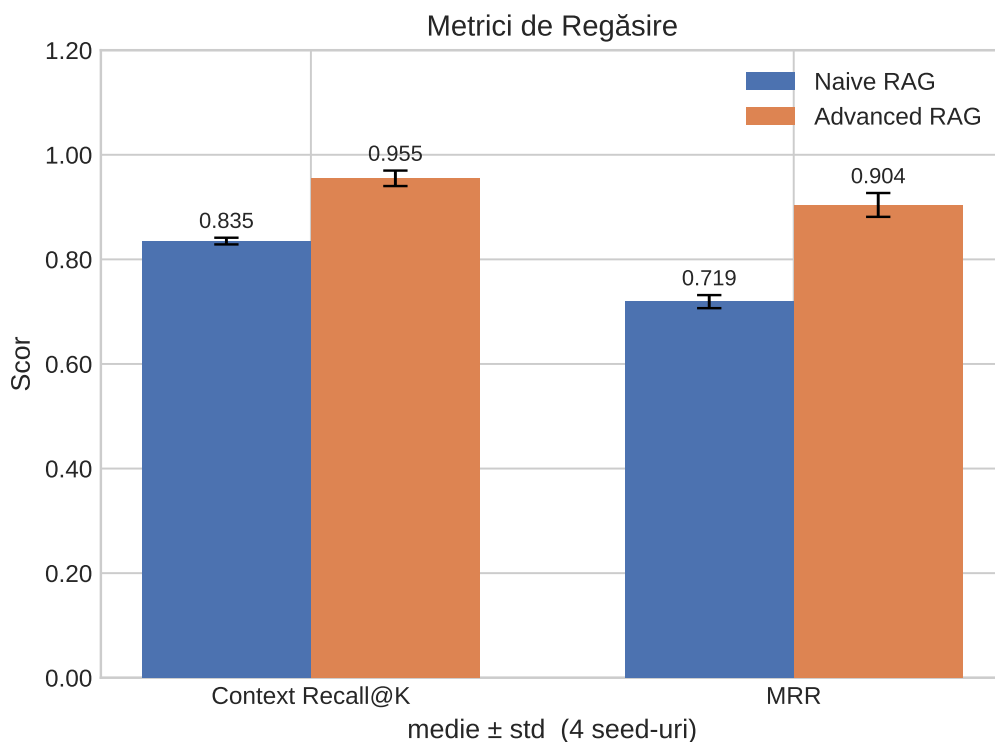


Figura 2.6: Metrici de recuperare: Context Recall@5 și MRR pentru Naive RAG și Advanced RAG. Barele de eroare reprezintă deviația standard calculată pe cele 4 seed-uri.

2.2.4 Semnificație statistică

Toate diferențele observate sunt semnificative statistic la $p < 0,05$, cu marje considerabile față de prag (Tabelul 2.4).

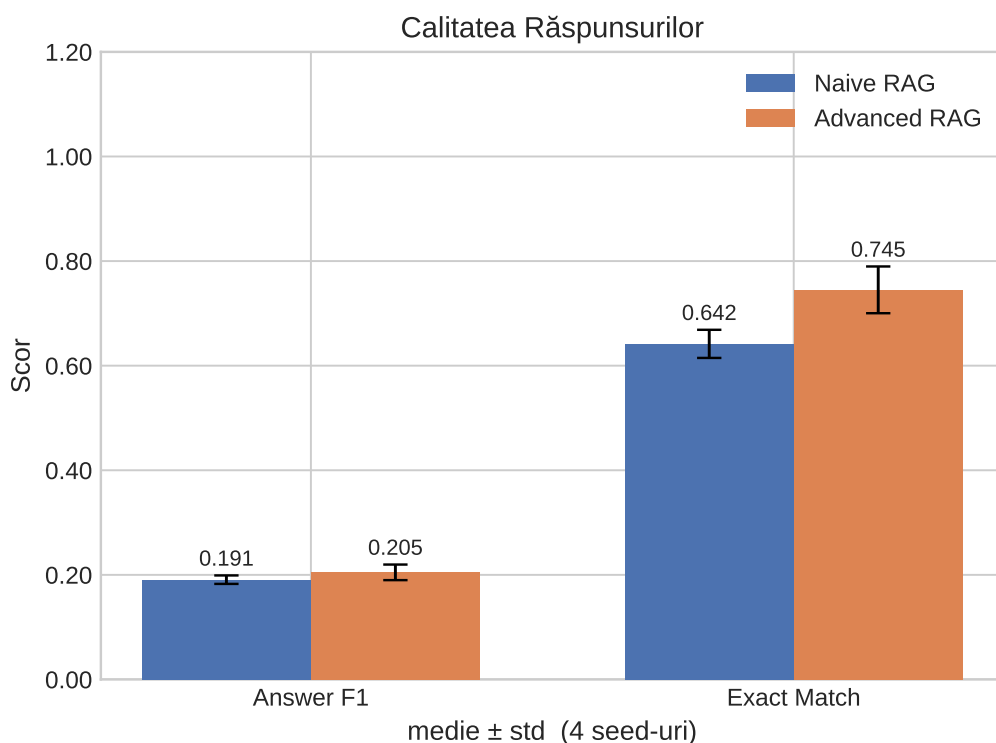


Figura 2.7: Calitatea răspunsurilor: Answer F1 și Exact Match pentru Naive RAG și Advanced RAG.

Tabel 2.4: Teste de semnificație statistică (Advanced RAG vs. Naive RAG, $N = 600$ perechi)

Metrică	Test	Valoarea p	Observații
MRR	Wilcoxon	$6,5 \times 10^{-27}$	
Answer F1	Wilcoxon	$1,8 \times 10^{-3}$	
Context Recall	McNemar	$7,7 \times 10^{-20}$	Adv. mai bun: 74; Naive mai bun: 2
Exact Match	McNemar	$4,3 \times 10^{-10}$	Adv. mai bun: 82; Naive mai bun: 20

Valoarea p extrem de mică pentru MRR ($6,5 \times 10^{-27}$) indică o diferență sistematică, nu un artefact al eșantionării. Testul McNemar pentru Context Recall arată că există 74 de întrebări unde Advanced RAG recuperează fragmentul corect, iar Naive RAG nu reușește, față de doar 2 în situația inversă. Raportul de 37:1 în favoarea sistemului avansat confirmă că îmbunătățirile în recuperare nu provin din cazuri marginale, ci reprezintă un avantaj consistent.

2.2.5 Stabilitatea rezultatelor pe seed-uri

Pentru a verifica dacă diferențele nu depind de un eșantion particular de întrebări, evaluarea a fost repetată pe patru seed-uri, fiecare selectând alte 150 de întrebări. Tabelul 2.5 prezintă valorile pentru Context Recall și MRR pe fiecare seed.

Se observă că Advanced RAG depășește Naive RAG pe fiecare seed în parte, nu doar în medie. Variația între seed-uri este mică pentru ambele sisteme (sub 2 puncte procentuale la Context Recall pentru Naive și sub 3,5 pentru Advanced), ceea ce arată că rezultatele sunt stabile și nu depind de un eșantion norocos. Această consistență întărește concluzia testelor de semnificație: avantajul arhitecturii avansate este real și reproductibil.

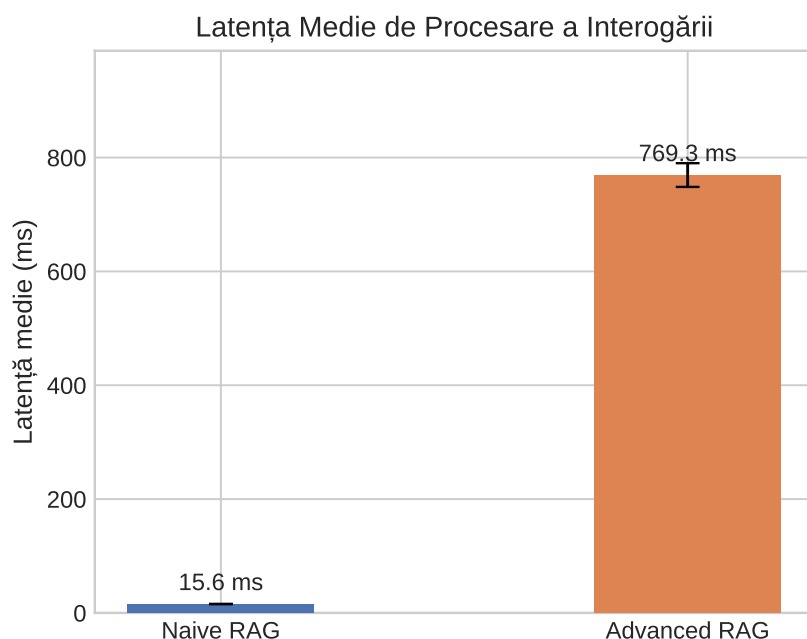


Figura 2.8: Latența medie de procesare a interogării în milisecunde. Latența Advanced RAG include apelul LLM pentru expansiunea interogării, recuperarea hibridă paralelă și re-scorarea cu cross-encoder.

Tabel 2.5: Context Recall@5 și MRR pe fiecare dintre cele 4 seed-uri (150 de întrebări per seed)

Seed	Context Recall@5		MRR	
	Naive	Advanced	Naive	Advanced
42	0,833	0,960	0,721	0,915
1042	0,840	0,947	0,728	0,898
2042	0,827	0,940	0,701	0,875
3042	0,840	0,973	0,727	0,928

2.3 Analiză calitativă

2.3.1 Compromisul dintre calitate și latență

Cel mai vizibil cost al arhitecturii avansate este latența de recuperare: 769 ms față de 16 ms pentru varianta de bază, o creștere de aproximativ 50 de ori, măsurată pe configurația hardware din Secțiunea 2.1.4. Această diferență are câteva surse distincte. Expansiunea interogărilor implică un apel LLM (Qwen3-1.7B), care durează câteva sute de milisecunde chiar și pentru un model de 1.7B parametri. Recuperarea densă și BM25 rulează în paralel pe toate cele trei formulări (originală plus două alternative), ceea ce limitează parțial impactul, dar cross-encoder-ul introduce o latență suplimentară pentru re-scorarea celor 40 de candidați.

Defalcarea acestei latențe este instructivă. Componenta dominantă este apelul la modelul de expansiune a interogărilor, care, deși folosește un model mic, presupune o generare completă și consumă cea mai mare parte din cele 770 ms. Recuperarea densă și BM25 rulează în paralel pe cele trei formulări și contribuie relativ puțin, fiind operații de căutare optimizate. Rerankingul cu cross-encoder adaugă un cost intermediar, proporțional cu numărul de candidați evaluați, în cazul de față 40. Naive RAG evită toate aceste etape, ceea ce explică latența sa de doar 16 ms, formată aproape integral din codificarea interogării și căutarea cosinus.

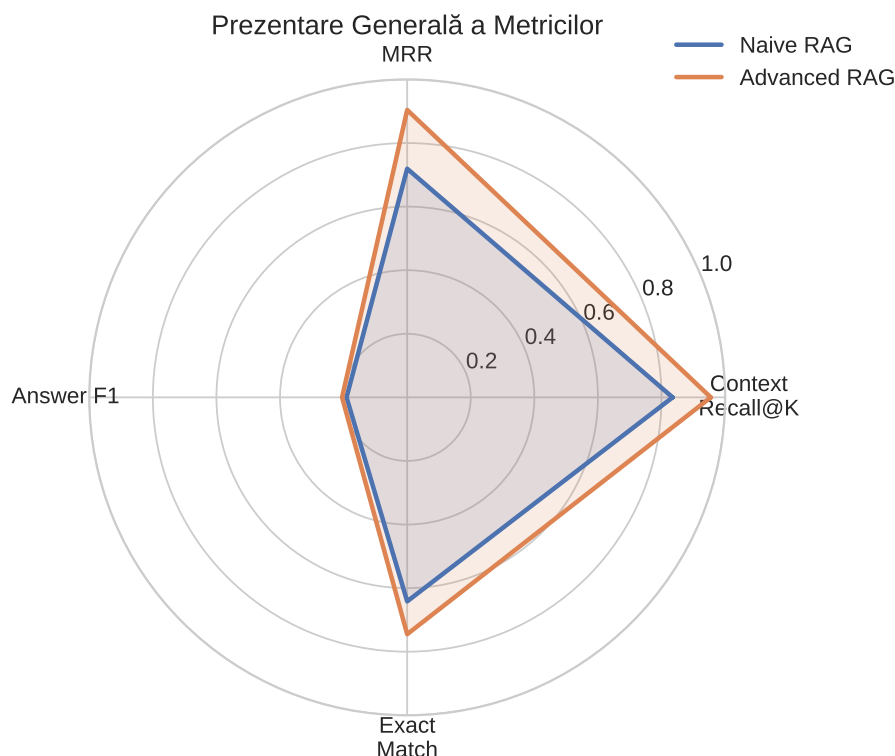


Figura 2.9: Vedere de ansamblu comparativă a metricilor de calitate (Context Recall@5, MRR, Answer F1, Exact Match). Latența nu este inclusă deoarece se află pe o scară diferită, în milisecunde față de intervalul de la 0 la 1.

În contextul aplicațiilor RAG reale, o latență de recuperare de 770 ms rămâne acceptabilă, deoarece generarea unui răspuns cu un model de 8B parametri durează oricum mai multe secunde. Totuși, în scenarii unde se doresc timpi de răspuns sub o secundă, overhead-ul expansiunii interogărilor devine o constrângere reală și ar putea fi eliminat printr-o versiune fără expansiune, care combină recuperarea hibridă fără reformulare [2]. Această observație sugerează că expansiunea interogărilor ar trebui activată selectiv, doar pentru interogările unde aduce un câștig real de recall, nu pentru fiecare cerere.

2.3.2 Unde ajută recuperarea hibridă

Diferența cea mai pronunțată dintre cele două sisteme apare la metricile de recuperare, nu la cele de generare. Creșterea cu 12,0 puncte la Context Recall înseamnă că, în 12 din 100 de cazuri unde Naive RAG eșuează, Advanced RAG găsește totuși fragmentul corect. MRR crescut de la 0,719 la 0,904 arată că, pe lângă faptul că găsește mai des fragmentul corect, îl și plasează pe poziții mai înalte.

Recuperarea hibridă (densă + BM25) are un avantaj clar pe interogările cu termeni specifici: date, numere, denumiri proprii, acronime. Modelul de embedding codifică înțeles semantic, dar poate dilua termeni rari într-un spațiu vectorial continuu unde similaritatea cosinus favorizează documente tematic apropiate, nu neapărat lexical identice. BM25 [8] nu are această problemă: un termen care apare exact o dată în corpus va fi regăsit cu precizie dacă apare și în interogare. Expansiunea interogărilor contribuie la recall prin acoperirea variantelor de formulare, reducând dependența de formularea exactă a întrebării [2].

2.3.3 Decalajul dintre recuperare și generare

Îmbunătățirile la recuperare nu se traduc proporțional în îmbunătățiri la generare. Context Recall crește cu 12 puncte procentuale, dar Answer F1 crește cu doar 7% relativ, iar Exact Match cu 10 puncte procentuale.

Explicația principală este că SQuAD a fost conceput ca benchmark pentru citire extractivă, unde modelul ar trebui să extragă textual un fragment din pasaj. Modelele generative moderne tind să parafrazeze răspunsul mai degrabă decât să îl reproducă cuvânt cu cuvânt, ceea ce scade Answer F1 față de valoarea maximă posibilă, chiar dacă răspunsul este corect semantic. O a doua explicație este că Naive RAG recuperează corect fragmentul în 83,5% din cazuri, ceea ce înseamnă că modelul generativ primea oricum contextul corect pentru cea mai mare parte a întrebărilor. Câștigul din Advanced RAG vine tocmai din cazurile dificile, adică interogările cu termeni rari sau cu răspunsuri scurte greu de localizat prin similaritate semantică pură, unde generarea oricum eșuează mai des.

2.3.4 Constrângerea de grounding și refuzurile de a răspunde

Ambele pipeline-uri folosesc un prompt de sistem care constrânge modelul să răspundă exclusiv din contextul furnizat și să refuze explicit dacă informația lipsește. Exact promptul de refuz este: *“The provided context does not contain enough information to answer this question.”* Această decizie de proiectare este direct motivată de principiul fidelității contextuale descris în [10]: un sistem care preferă să refuze să răspundă, decât să inventeze, este mai fiabil decât unul care generează răspunsuri plauzibile dar incorecte.

Refuzurile nu pot fi evaluate cu Answer F1 sau Exact Match, deoarece nu conțin conținut factual comparabil cu referința. Advanced RAG generează mai puține refuzuri decât Naive RAG, pentru că recuperează mai frecvent fragmentul corect și îl plasează pe primele poziții, astfel că modelul primește mai rar un context din care nu poate răspunde.

2.3.5 Observații calitative din interfața Streamlit

Interfața Streamlit permite observarea directă a diferențelor de comportament fără a parcurge metricile agregate.

Pe întrebări cu termeni numerici sau coduri specifice (“Câte specii...”, “În ce an...”, “Care este codul...”), Naive RAG returnează adesea fragmente tematic relevante care nu conțin numărul exact. Advanced RAG găsește fragmentul corect prin BM25, care tratează numerele ca termeni distincți și nu le diluează în spațiu vectorial.

Când răspunsul nu există în corpus, ambele sisteme refuză explicit. Diferența este că Advanced RAG produce mai rar răspunsuri parțiale construite din context adiacent irelevant, deoarece cross-encoder-ul elimină candidații slab relevanți înainte de generare.

Pe întrebări tematice generale (“Despre ce este...”, “Care este rolul...”), ambele sisteme produc rezultate similare. Recuperarea densă funcționează bine pe similitudine semantică, iar avantajul hibridizării este mai puțin pronunțat în aceste cazuri.

Aceste observații sunt consistente cu rezultatele cantitative: câștigurile Advanced RAG sunt mai mari pe cazurile dificile (termeni rari, răspunsuri numerice) și mai mici pe interogările tematice generale, unde recuperarea densă funcționează deja bine [2, 8].

2.3.6 Analiza erorilor

Deși Advanced RAG depășește Naive RAG în ansamblu, testul McNemar arată că există și cazuri în care varianta de bază răspunde corect, iar cea avansată nu. La Exact Match, 20 de

întrebări sunt rezolvate de Naive RAG, dar ratate de Advanced RAG, față de 82 în situația inversă. La Context Recall, raportul este de doar 2 la 74. Aceste cazuri minoritare merită analizate, deoarece arată limitele optimizărilor.

O primă sursă de erori ține de expansiunea interogărilor. Atunci când modelul de reformulare produce variante care se îndepărtează de sensul original, recuperarea hibridă poate aduce fragmente relevante pentru reformulare, dar nu pentru întrebarea reală. În aceste situații, simplitatea recuperării dense a Naive RAG, care lucrează direct cu interogarea originală, devine un avantaj. Acest fenomen explică o parte din cele 20 de cazuri unde Naive RAG obține Exact Match, iar Advanced RAG nu.

O a doua sursă ține de reranking. Cross-encoderul reordonează fragmentele după relevanța estimată, dar estimarea nu este perfectă: ocazional, un fragment care conține răspunsul exact este plasat sub pragul de cinci fragmente, în timp ce recuperarea densă simplă l-ar fi păstrat. Aceste erori sunt rare, dar reale, și ilustrează că o etapă suplimentară de procesare nu garantează întotdeauna un rezultat mai bun pe fiecare interogare în parte.

A treia categorie ține de natura metricii Exact Match. O parte din diferențele observate nu reflectă răspunsuri greșite, ci reformulări: ambele sisteme pot produce un răspuns corect semantic, dar care nu se potrivește exact cu referința. În aceste cazuri, diferența la Exact Match este un artefact al metricii, nu o eroare reală de extragere. Această observație întărește argumentul că o evaluare completă necesită și metrici care tolerează reformularea, precum cele bazate pe fidelitate.

2.3.7 Limitările demonstratorului

Rezultatele trebuie interpretate în lumina câtorva limitări ale configurației experimentale.

Prima ține de setul de date. SQuAD este un benchmark de tip QA extractiv, în care răspunsul este un fragment de text, nu o relație structurată. Deși permite o evaluare automată și precisă a recuperării, nu măsoară direct capacitatea de extragere a relațiilor între entități, care este scopul final al lucrării. Concluziile despre avantajul Advanced RAG se referă, strict, la recuperarea și răspunsul la întrebări, și urmează să fie confirmate pe seturi dedicate extragerii relațiilor.

A doua ține de scala evaluării. Cele 600 de întrebări per sistem oferă o bază statistică solidă, dar corpusul de paragrafe provine dintr-un singur set de date, omogen ca stil și domeniu. Pe corpusuri mai mari și mai diverse, comportamentul celor două sisteme ar putea diferi, în special în privința latenței și a calității recuperării.

A treia ține de evaluarea calității răspunsului. Metricile folosite (Exact Match și Answer F1) măsoară potrivirea cu referința, dar nu și fidelitatea răspunsului față de contextul recuperat. Evaluarea cu metrici de fidelitate, prevăzută ca extensie, ar oferi o imagine mai completă, mai ales pentru distincția dintre un răspuns corect și un răspuns plauzibil dar nefundamentat.

Concluzii

Rezultate obținute

Lucrarea a urmărit două obiective paralele: un studiu al literaturii de specialitate privind sistemele de tip Retrieval-Augmented Generation (RAG) și implementarea unui demonstrator care compară o arhitectură de bază cu una avansată pe un set de date public.

Din studiul de literatură (Capitolul 1) au reieșit câteva concluzii clare. Problema fundamentală pe care o adresează RAG rămâne aceeași ca la apariția termenului în 2020 [5]: modelele lingvistice mari au cunoștințe fixate la momentul antrenării și generează răspunsuri incorecte atunci când nu dețin informația solicitată. RAG rezolvă această limitare nu prin reantrenare, ci prin recuperarea documentelor relevante la momentul interogării și condiționarea generării pe aceste documente. Diversitatea abordărilor din literatură, de la Naive RAG la arhitecturi modulare și grafuri de cunoștințe [2, 6], reflectă că nu există o soluție universală: alegerea arhitecturii depinde de tipul sursei de date, de latența acceptabilă și de natura sarcinii.

Studiul a identificat că metodele de recuperare hibridă (densă + BM25 [8]) și reranking-ul cu cross-encoder aduc cele mai consistente îmbunătățiri față de recuperarea densă pură, în special pe interogări cu termeni rari sau numerici. Metodele adaptive precum Self-RAG [10] și CRAG [18] adaugă un nivel de auto-evaluare a calității recuperării, reducând halucinațiile cauzate de documente irelevante. Pentru extragerea relațiilor între entități, abordările bazate pe grafuri de cunoștințe [21] sunt cele mai potrivite structural, deoarece relațiile sunt reprezentate explicit și pot fi traversate direct.

Analiza metodelor a arătat și o tendință de evoluție a domeniului: de la pipeline-uri liniare fixe, spre sisteme care evaluează și corectează adaptiv calitatea recuperării, și mai departe spre arhitecturi care structurează explicit cunoștințele sub formă de graf. Fiecare pas pe acest drum aduce un câștig de capacitate, dar și un cost suplimentar de complexitate și de calcul. Niciuna dintre metode nu domină pe toate criteriile; alegerea potrivită depinde de tipul datelor, de latența acceptabilă și de natura sarcinii. Această observație justifică abordarea adoptată în lucrare, de a porni de la o arhitectură simplă și de a măsura efectul fiecărei optimizări, în loc de a implementa direct cea mai complexă variantă.

Din evaluarea experimentală (Capitolul 2) rezultatele arată că Advanced RAG depășește Naive RAG pe toate metricile de calitate, cu diferențe semnificative statistic ($p \ll 0,05$ în toate testele). Context Recall@5 a crescut de la 83,5% la 95,5% (+12,0 puncte procentuale), MRR de la 0,719 la 0,904 (+0,185), iar Exact Match de la 64,2% la 74,5% (+10,3 puncte procentuale). Testul McNemar pentru Context Recall a arătat că există 74 de întrebări unde Advanced RAG recuperează fragmentul corect iar Naive RAG nu, față de doar 2 în situația inversă, un raport de 37:1 în favoarea arhitecturii avansate.

Costul acestor îmbunătățiri este latența: Advanced RAG necesită 770 ms per interogare față de 16 ms pentru Naive RAG, o diferență de aproximativ 50 de ori. Cea mai mare parte a acestui overhead provine din apelul LLM pentru expansiunea interogărilor. În aplicații unde latența de recuperare poate fi ascunsă parțial de latența de generare (care durează oricum câteva secunde pentru modele de 8B parametri), compromisul este acceptabil. Îmbunătățirile la Answer F1 au fost mai modeste (7,3% relativ), ceea ce este de așteptat: SQuAD este un benchmark extractiv, iar modelele generative modifică formularea răspunsului chiar și atunci când contextul corect este disponibil.

Observațiile calitative din interfața comparativă au confirmat ceea ce arătau deja cifrele. Avantajul recuperării hibride s-a văzut cel mai clar pe întrebările cu termeni numerici și denumiri rare, unde recuperarea densă a variantei de bază aducea fragmente tematic apropiate, dar fără valoarea exactă cerută. Pe întrebările tematice generale, cele două sisteme s-au comportat similar, ceea ce arată că optimizările contează mai ales pe cazurile dificile.

Analiza pe seed-uri și testele de semnificație confirmă că aceste rezultate sunt stabile și reproductibile, nu artefacte ale unui eșantion particular. Advanced RAG depășește varianta de bază pe fiecare seed în parte, iar analiza erorilor arată că cele câteva cazuri în care Naive RAG răspunde mai bine se explică prin reformulări nereușite ale interogării sau prin limitele metricii Exact Match, nu printr-un avantaj real al recuperării simple. În ansamblu, studiul de literatură și partea experimentală conduc la aceeași concluzie: în sistemele RAG, calitatea recuperării este pârghia principală asupra calității răspunsului.

Contribuții originale

Contribuția principală a lucrării este demonstratorul implementat, disponibil ca proiect open-source, care compară Naive RAG și Advanced RAG pe același set de date și același model lingvistic, izolând contribuția etapelor de recuperare. Sistemul rulează local prin Ollama și ChromaDB, fără dependență de servicii externe.

Din punct de vedere tehnic, Advanced RAG adaugă patru optimizări față de varianta de bază. Recuperarea hibridă combină recuperarea densă (all-MiniLM-L6-v2 [7] + ChromaDB) cu recuperarea sparse (BM25 [8] prin `bm25s`), listele rezultate fuzionând prin Reciprocal Rank Fusion [9]. Expansiunea interogărilor folosește Qwen3-1.7B pentru a genera două reformulări ale interogării originale, reducând dependența de formularea exactă. Reranking-ul cu cross-encoder (`ms-marco-MiniLM-L-6-v2`) re-scorează cele mai bune 40 de candidate și returnează primele 5. Ambele sisteme refuză explicit când contextul recuperat nu conține informația solicitată, în loc să genereze răspunsuri plauzibile dar incorecte [10].

O contribuție separată este modulul de evaluare automată, care rulează ambele sisteme pe aceleași întrebări, pe mai multe seed-uri, cu checkpoint salvat atomic după fiecare rulare, astfel încât o întrerupere să nu piardă rezultatele deja calculate. Modulul aplică testele de semnificație statistică (Wilcoxon și McNemar) pe perechile de rezultate, exclude refuzurile corecte de la metrica de relevanță pentru a nu penaliza sistemul mai prudent și generează automat figurile comparative folosite în lucrare. Interfața Streamlit completează evaluarea automată cu testare interactivă: ambele sisteme răspund simultan la aceeași întrebare, iar răspunsurile, fragmentele recuperate și timpii de procesare apar unul lângă altul.

Taxonomia din Capitolul 1 a fost adaptată pentru extragerea relațiilor între entități, nu pentru domeniul RAG în ansamblu, ceea ce limitează acoperirea dar face clasificarea mai directă pentru înțelegerea alegerii arhitecturale din demonstrator.

Dincolo de cifrele de performanță, demonstratorul are valoare ca instrument de comparație: pentru că ambele sisteme rulează pe aceeași bază și pot fi interogate simultan, diferențele de comportament devin vizibile direct, nu doar prin metrici agregate. Interfața permite observarea felului în care recuperarea hibridă recuperează termeni pe care recuperarea densă îi ratează, sau a felului în care constrângerea de groundare conduce la un refuz explicit în locul unei halucinații. Această transparență face din demonstrator un punct de plecare util atât pentru extinderea către extragerea relațiilor, cât și pentru înțelegerea practică a compromisurilor descrise teoretic în studiul de literatură.

Perspective de dezvoltare ulterioară

O limitare concretă a evaluării actuale este că s-a realizat pe SQuAD, un set de date de tip QA extractiv. Lucrarea urmărește aplicarea RAG la extragerea relațiilor între entități, pentru care T-REx [35] (aliniament Wikipedia-Wikidata) sau DocRED [29] sunt mai potrivite structural, deoarece conțin triplete (subiect, predicat, obiect) explicit adnotate. Adaptarea benchmark-ului pentru aceste seturi rămâne pasul imediat următor.

Evaluarea RAGAS [27], cu metrici de fidelitate (faithfulness) și relevanță a răspunsului (answer relevancy), nu a putut fi finalizată în limitele de timp disponibile din cauza costului computațional ridicat (modelul judecător rulează câte un apel LLM per pereche întrebare-răspuns). O evaluare completă ar adăuga perspectiva calității fundamentării în context, independentă de potrivirea textuală cu referința.

Pe latura arhitecturală, GraphRAG [21] și HippoRAG [22] indexează explicit relațiile dintre entități și le utilizează la recuperare, ceea ce le face candidați direcți pentru extragerea relațiilor față de pipeline-urile actuale bazate pe fragmente de text. O comparație extinsă cu aceste metode ar clarifica în ce măsură structura relațională explicită aduce beneficii față de recuperarea hibridă pe text nestructurat.

Latența Advanced RAG (770 ms per interogare) provine în cea mai mare parte din apelul LLM pentru expansiunea interogărilor. O variantă fără expansiune, care păstrează recuperarea hibridă și cross-encoder-ul, ar reduce latența la câteva zeci de milisecunde cu o pierdere relativ mică la recall. Această variantă ar fi utilă în scenarii cu constrângeri stricte de timp de răspuns.

Evaluarea actuală folosește Qwen3-8B ca model generativ, fără fine-tuning. Modele mai mari sau antrenate specific pe extragerea relațiilor pot produce îmbunătățiri la Answer F1 independent de arhitectura de recuperare. Separarea contribuției modelului generativ de contribuția pipeline-ului de recuperare rămâne o direcție deschisă pentru cercetare ulterioară.

O direcție complementară privește extinderea evaluării dincolo de limba engleză. SQuAD și majoritatea seturilor de date folosite sunt în engleză, iar comportamentul sistemului pe texte în alte limbi, inclusiv în română, rămâne neexplorat. Modelul de embedding și cel generativ folosite au capacități multilingve, dar calitatea recuperării și a extragerii relațiilor pe corpusuri în limba română ar trebui măsurată separat, deoarece resursele și performanța modelelor diferă semnificativ între limbi. O astfel de evaluare ar fi relevantă pentru aplicarea sistemului pe documente locale, unde nevoia de extragere a relațiilor este la fel de prezentă.

În fine, trecerea de la demonstrator la un sistem utilizabil în practică ar cere câteva completări ingineresti. Corpusul ar trebui extins de la paragrafele SQuAD la documente reale ale aplicației vizate, cu o etapă de fragmentare adaptată formatului acestora. Indexul ar trebui actualizat incremental pe măsură ce apar documente noi, fără reindexarea întregii colecții. Pentru mai mulți utilizatori simultani, expansiunea interogărilor și rerankingul ar trebui servite din instanțe dedicate, deoarece partajarea unei singure plăci grafice între generare și recuperare devine un punct de strangulare. Aceste completări nu schimbă arhitectura validată în lucrare, însă fără ele sistemul nu poate fi pus în fața unor utilizatori reali.

Pe scurt, lucrarea confirmă pe un caz concret o observație recurentă din literatură: în sistemele RAG, calitatea recuperării este factorul determinant pentru calitatea răspunsului final. Optimizările aduse de Advanced RAG, deși nu sunt complexe, produc îmbunătățiri consistente și semnificative statistic, la un cost de latență acceptabil pentru cele mai multe aplicații. Această concluzie justifică direcția aleasă și oferă o bază solidă pentru pasul următor, aplicarea și evaluarea aceleiași arhitecturi pe extragerea relațiilor între entități, scopul final al cercetării.

Pe lângă rezultatele în sine, lucrarea s-a sprijinit pe o metodologie strictă de comparație: aceeași bază de date, același model generativ, aceleași întrebări și teste statistice pereche. Fără aceste precauții, diferența dintre cele două arhitecturi ar fi putut fi atribuită la fel de bine

întâmplării sau unor factori colaterali. Deciziile practice luate pe parcurs, cum sunt alegerea modelelor potrivite pentru fiecare etapă sau excluderea refuzurilor corecte din evaluare, sunt și ele un rezultat al lucrării și un punct de plecare pentru extinderile descrise mai sus.

Bibliografie

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. doi: 10.48550/arXiv.1706.03762.
- [2] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-Augmented Generation for Large Language Models: A survey, 2024.
- [3] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553): 436–444, 2015. doi: 10.1038/nature14539.
- [4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [6] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph Retrieval-Augmented Generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024. doi: 10.48550/arXiv.2408.08921.
- [7] Vladimir Karpukhin, Barlas Öğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020. doi: 10.18653/v1/2020.emnlp-main.550.
- [8] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019.
- [9] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114.
- [10] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*, 2023.
- [11] Bowen Jin, Jinsung Yoon, Jiawei Han, and Serkan O Arik. Long-context LLMs meet RAG: Overcoming challenges for long inputs in RAG. *arXiv preprint arXiv:2410.05983*, 2024. doi: 10.48550/arXiv.2410.05983.

- [12] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020. doi: 10.1145/3397271.3401075.
- [13] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1762–1777, 2023. doi: 10.18653/v1/2023.acl-long.99.
- [14] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-Augmented language model pre-training. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 3929–3938, 2020.
- [15] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 874–880, 2021. doi: 10.18653/v1/2021.eacl-main.74.
- [16] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, et al. Improving language models by retrieving from trillions of tokens. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 2206–2240, 2022.
- [17] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024.
- [18] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024. doi: 10.48550/arXiv.2401.15884.
- [19] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 7036–7050, 2024. doi: 10.18653/v1/2024.naacl-long.389.
- [20] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7969–7992, 2023. doi: 10.18653/v1/2023.emnlp-main.495.
- [21] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. doi: 10.48550/arXiv.2404.16130.
- [22] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hippo-RAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [23] Lang Mei, Siyu Mo, Zhihan Yang, and Chong Chen. A survey of multimodal retrieval-augmented generation. *arXiv preprint arXiv:2504.08748*, 2025. doi: 10.48550/arXiv.2504.08748.

- [24] Peng Xia, Kangyu Zhu, Haoran Li, Tianze Wang, Weijia Shi, Sheng Wang, Linjun Zhang, James Zou, and Huaxiu Yao. MMed-RAG: Versatile multimodal RAG system for medical vision language models. *arXiv preprint arXiv:2410.13085*, 2024. doi: 10.48550/arXiv.2410.13085.
- [25] Haoran Hao, Jiaming Han, Changsheng Li, Yu-Feng Li, and Xiangyu Yue. RAP: Retrieval-augmented personalization for multimodal large language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14538–14548, 2025. doi: 10.1109/CVPR52734.2025.01355.
- [26] Jun Chen, Dannong Xu, Junjie Fei, Chun-Mei Feng, and Mohamed Elhoseiny. Document haystacks: Vision-language reasoning over piles of 1000+ documents. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 24817–24826, 2025. doi: 10.1109/CVPR52734.2025.02311.
- [27] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217*, 2023. doi: 10.48550/arXiv.2309.15217.
- [28] Qingyu Tan, Lu Xu, Lidong Bing, Hwee Tou Ng, and Sharifah Mahani Aljunied. Revisiting DocRED – addressing the false negative problem in relation extraction. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8472–8487, 2022. doi: 10.18653/v1/2022.emnlp-main.580.
- [29] Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou, and Maosong Sun. DocRED: A large-scale document-level relation extraction dataset. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 764–777, 2019. doi: 10.18653/v1/P19-1074.
- [30] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1264.
- [31] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. Natural Questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics (TACL)*, 7:453–466, 2019. doi: 10.1162/tacl.a_00276.
- [32] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1601–1611, 2017. doi: 10.18653/v1/P17-1147.
- [33] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380, 2018. doi: 10.18653/v1/D18-1259.
- [34] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, et al. CRAG – comprehensive RAG benchmark. In *Advances in Neural Information Processing Systems (NeurIPS) – Datasets and Benchmarks Track*, 2024.

- [35] Hady Elsahar, Pavlos Vougiouklis, Arslan Remaci, Christophe Gravier, Jonathon Hare, Frederique Laforest, and Elena Simperl. T-REx: A large scale alignment of natural language with knowledge base triples. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*, pages 3448–3452, 2018.
- [36] Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. Creating training corpora for NLG micro-planners. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 179–188, 2017. doi: 10.18653/v1/P17-1017.
- [37] Iris Hendrickx, Su Nam Kim, Zornitsa Kozareva, Preslav Nakov, Diarmuid Ó Séaghdha, Sebastian Padó, Marco Pennacchiotti, Lorenza Romano, and Stan Szpakowicz. SemEval-2010 task 8: Multi-way classification of semantic relations between pairs of nominals. In *Proceedings of the 5th International Workshop on Semantic Evaluation (SemEval)*, pages 33–38, 2010.
- [38] Yuhao Zhang, Victor Zhong, Danqi Chen, Gabor Angeli, and Christopher D. Manning. Position-aware attention and supervised data improve slot filling. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 35–45, 2017. doi: 10.18653/v1/D17-1004.